

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-ab7951d47a4a465a7.jsonl`  
- SHA-256: `143016c9dcccfd58ab399df6ef499226591b3fcd02437d562d6ded3a276e8a87`  
- Source modified: `2026-06-18T07:50:33+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_e5140909-890`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T07:47:49.194Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base  
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.  
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong  
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,  
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +  
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.  
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.  
AREA: backend/main.py, backend/infrastructure/database.py, backend/config/, backend/utils/,  
backend/providers/. API request validation, DB transactions/migrations, config defaults,  
hashing, provider error/retry handling.
```

2. assistant (2026-06-18T07:47:51.663Z)

I'll analyze the specified areas for real, reachable defects. Let me start by reading the target files.

3. user (2026-06-18T07:47:53.085Z)

```
1      import os  
2      import sys  
3      import argparse  
4      import logging  
5      import threading  
6      import traceback  
7      import uuid  
8      from concurrent.futures import ThreadPoolExecutor  
9      import uvicorn  
10     from dotenv import load_dotenv  
11  
12     load_dotenv()  
13  
14     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print  
for direct user-facing messages).  
15     logging.basicConfig(  
16         level=logging.INFO,  
17         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',  
18         datefmt='%H:%M:%S'  
19     )  
20     logger = logging.getLogger(__name__)  
21     from fastapi import FastAPI, HTTPException, Request  
22     from fastapi.middleware.cors import CORSMiddleware  
23     from pydantic import BaseModel  
24     from typing import List, Dict, Any, Optional
```

```

25
26 from fastapi.staticfiles import StaticFiles
27 from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28 from backend.pipeline.validators.base import registry
29 from backend.infrastructure.database import Database
30 from backend.pipeline.segmenters.base import segmenter_registry
31 from backend.pipeline.stitcher.compiler import VideoCompiler
32 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
33 from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34 from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
35 from backend.results import Failure # standardized error/result contract
36 from backend.reporting import emit_indexer_report
37
38 app = FastAPI(title="Sports Highlight Indexer API")
39
40 # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
41 # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42 # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43 app.add_middleware(
44     CORSMiddleware,
45     allow_origins=["*"],
46     allow_credentials=False,
47     allow_methods=["*"],
48     allow_headers=["*"],
49 )
50
51 # Serves static frontend files directly (now under api/ per approved structure)
52 os.makedirs("backend/api/static", exist_ok=True)
53 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55 @app.get("/", response_class=HTMLResponse)
56 def serve_index():
57     index_path = "backend/api/static/index.html"
58     if os.path.exists(index_path):
59         with open(index_path, "r", encoding="utf-8") as f:
60             return f.read()
61     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63 # Global variables initialized at startup
64 db_instance: Optional[Database] = None
65 global_config: Optional[AppConfig] = None # now typed (Pydantic model)
66
67 # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68 # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69 # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70 # already parallelize their AI handoff internally via their own pools.
71 _job_executor: Optional[ThreadPoolExecutor] = None
72
73 def _get_executor() -> ThreadPoolExecutor:
74     """Lazy module-global executor; tests monkeypatch this for inline execution."""
75     global _job_executor
76     if _job_executor is None:
77         _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78     return _job_executor
79
80 # Legacy thin wrapper for any remaining direct calls during transition.
81 # New code should import load_app_config / AppConfig from backend.config directly.
82 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83     cfg = load_app_config(config_path)
84     return cfg.model_dump(mode="json")

```

```

85
86 # --- API Models ---
87 class ProcessRequest(BaseModel):
88     video_path: str
89     player_pool: Optional[List[str]] = None
90     max_frames: Optional[int] = None
91     segmenter: Optional[str] = None
92
93 class QueryRequest(BaseModel):
94     video_id: str
95     server: Optional[str] = None
96     receiver: Optional[str] = None
97     duration_min: Optional[float] = None
98     event_type: Optional[str] = None
99
100 class CompileRequest(BaseModel):
101     video_id: str
102     segment_ids: List[int]
103     output_filename: str
104
105 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
110     and
111     keep the prior good results intact ? a failed/empty re-run never destroys prior
112     data.
113     """
114     if segments:
115         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
116     else:
117         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
118
119 # --- API Endpoints ---
120 @app.get("/api/config")
121 def get_config():
122     return global_config
123
124 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
125     """The full hash ? validate ? segment ? report pipeline for one video.
126
127     This is the former synchronous /api/process body, unchanged in behavior, now run on
128     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
129     the sync endpoint used to return (UI compatibility); the per-video observability
130     report is still emitted in `finally:` and grafted as response["reports"].
131
132     `progress` is an optional callable(stage: str) used to surface coarse job progress.
133     Raises on unexpected internal errors ? the caller records those as a job failure
134     (after this function has already restored the pre-run segments, see below).
135     """
136     notify = progress or (lambda stage: None)
137
138     notify("hashing")
139     video_id = compute_video_id(req.video_path)
140     was_reingest = db_instance.get_video(video_id) is not None
141
142     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
143     report)
144     notify("validating")
145     logger.info(f"Running validations for {req.video_path}...")
146     val_results, val_context = registry.run_all_with_context(req.video_path,
147     global_config)
148     failures = [r for r in val_results if not r.passed]

```

```

146
147     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
148     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
149 "motion")
150     seg_name = req.segmenter or default_seg
151     segmenter_actual = None
152     segmentation_ran = False
153     response: Optional[Dict[str, Any]] = None
154     try:
155         if failures:
156             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
157             # status; it no longer destroys the video's prior good segments.
158             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
159 for r in val_results])
160             response = {
161                 "video_id": video_id,
162                 "status": "failed",
163                 "message": "Video failed validation checks.",
164                 "results": [r.model_dump() for r in val_results],
165             }
166             return response
167
168     # 2. Run segmenter & indexer
169     logger.info("[API] Video passed checks. Segmenting and indexing...")
170     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
171 r in val_results])
172
173     segmenter_cls = segmenter_registry.get(seg_name)
174     if not segmenter_cls:
175         # Submit-time validation already 400s unknown names; this is the defensive
176         # in-worker path (e.g. config default pointing at an unregistered
177 segmenter).
178         available = list(segmenter_registry.list_available().keys())
179         response = {
180             "video_id": video_id,
181             "status": "failed",
182             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
183 {available}",
184             "results": [r.model_dump() for r in val_results],
185             "play_segments": [],
186         }
187         return response
188
189     logger.info(f"[API] Using segmenter: {seg_name}")
190     notify(f"segmenting ({seg_name})")
191     segmenter_actual = seg_name
192     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
193 only
194     # if the run yields segments; otherwise drop the new partial rows and keep the
195 old.
196     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
197     segmenter = segmenter_cls(db_instance, global_config)
198     try:
199         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
200 req.max_frames)
201     except Exception:
202         # A raising segmenter must not leave half-written rows: restore the pre-run
203         # state (same destroy-after-confirm contract as the Failure branch), then
204         let
205             # the job worker record the failure.
206             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
207             raise
208     segmentation_ran = True
209
210     if isinstance(result, Failure):

```

```

202         logger.error(f"[API] Segmenter failed: {result.message}")
203         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204         response = {
205             "video_id": video_id,
206             "status": "failed",
207             "message": f"Segmenter '{seg_name}' failed: {result.message}",
208             "results": [r.model_dump() for r in val_results],
209             "play_segments": [],
210         }
211         return response
212
213         segments = result.value if hasattr(result, "value") else result
214         notify("finalizing")
215         _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217         response = {
218             "video_id": video_id,
219             "status": "success",
220             "message": f"Successfully indexed {len(segments)} play segments using
221 '{seg_name}' segmenter.",
222             "results": [r.model_dump() for r in val_results],
223             "play_segments": segments,
224         }
225         return response
226     finally:
227         # Always emit the per-video observability report (next to the video, or to
228         # report.output_dir). Never raises. Surface the paths in the response.
229         paths = emit_indexer_report(
230             db=db_instance, video_id=video_id, video_path=req.video_path,
231             config=global_config,
232             val_results=val_results, val_context=val_context,
233             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
234             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
235         )
236         if paths and isinstance(response, dict):
237             response["reports"] = paths
238
239     class JobWaiting(Exception):
240         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
241         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
242         NOT a failure. The worker persists `next_poll_at` + `progress` (via
243         `set_job_waiting`),
244         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
245         date
246         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
247         contract.
248
249         The wiring is additive: the production segmenter path never raises this today (zero
250         behaviour change), so a job runs exactly as before. The hook is what a later slice
251         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
252         """
253
254         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
255             super().__init__(f"job parked for {delay_sec:.0f}s")
256             self.delay_sec = max(0.0, float(delay_sec))
257             self.progress = progress
258
259         def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
260             """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
261             re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
262             the original submit. The row stores exactly the ProcessRequest fields."""
263             return ProcessRequest(
264                 video_path=job["video_path"],

```

```

262         player_pool=job.get("player_pool"),
263         max_frames=job.get("max_frames"),
264         segmenter=job.get("segmenter"),
265     )
266
267
268 def _run_claimed_job(job: Dict[str, Any]) -> None:
269     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
270     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
271
272     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
273     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
275     job self-scheduled and will be re-claimed when `next_poll_at` is due.
276     """
277     job_id = job["id"]
278     req = _job_to_request(job)
279     try:
280         result = _execute_processing(
281             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282         if result.get("video_id"):
283             db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285     except JobWaiting as w:
286         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289     except Exception as e:
290         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294 def _drain_due_jobs() -> int:
295     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300     a single-worker box keeps making progress with no central timer (the DB is the
clock).
301     Returns the number of jobs that reached a terminal/parked state this tick.
302     """
303     ran = 0
304     while True:
305         job = db_instance.claim_due_job()
306         if job is None:
307             break
308         ran += 1
309         _run_claimed_job(job)
310     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312     # on the next submit/drain. Best-effort ? never raises into the worker.
313     _schedule_next_drain_if_waiting()
314     return ran
315
316
317 def _schedule_next_drain_if_waiting() -> None:

```

```

318     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319     self-scheduled job resumes with no further client submit. The drain itself re-checks
320     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321     try:
322         delay = db_instance.seconds_until_next_due()
323     except Exception as e: # pragma: no cover - defensive; never wedge the worker
324         logger.error(f"Could not compute next-due delay: {e}")
325         return
326     if delay is None:
327         return # nothing waiting
328     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
329     _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
336         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
337         timer.daemon = True
338         timer.start()
339
340
341     #: A parked job further out than this is re-checked by a capped wake rather than one
long
342     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
343     _MAX_DRAIN_WAKE_SEC = 60.0
344
345
346     @app.post("/api/process", status_code=202)
347     def process_video(req: ProcessRequest):
348         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
349
350         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
351         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
360         if not os.path.exists(req.video_path):
361             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
362         if req.segmenter and not segmenter_registry.get(req.segmenter):
363             available = list(segmenter_registry.list_available().keys())
364             raise HTTPException(
365                 status_code=400,
366                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
367             )
368
369         job_id = uuid.uuid4().hex
370         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
371                                     req.player_pool, req.max_frames):
372             raise HTTPException(status_code=500, detail="Failed to persist job record.")
373         _get_executor().submit(_drain_due_jobs)

```

```

374         return JSONResponse(
375             status_code=202,
376             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
377         )
378
379
380 @app.get("/api/jobs/{job_id}")
381 def get_job(job_id: str):
382     """Job state: {status, progress, result, reports}. `status` = execution state
383     (queued|running|done|failed); the pipeline verdict is result["status"]."""
384     job = db_instance.get_job(job_id)
385     if not job:
386         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
387     result = job.get("result")
388     return {
389         "job_id": job["id"],
390         "status": job["status"],
391         "progress": job.get("progress"),
392         "video_id": job.get("video_id"),
393         "error": job.get("error"),
394         "result": result,
395         "reports": (result or {}).get("reports"),
396         "created_at": job.get("created_at"),
397         "started_at": job.get("started_at"),
398         "finished_at": job.get("finished_at"),
399     }
400
401 # --- Chunked upload (B2, PR-2) -----
402 # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403 # proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
404 # under security.upload_dir, then submitted to /api/process by the returned path.
405 # Upload state is in-process by design (this is the single-box alpha): a server restart
406 # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407 # arrives with PR-3 identity scoping.
408
409 _uploads: Dict[str, Dict[str, Any]] = {}
410 _uploads_lock = threading.Lock()
411
412
413 class UploadInitRequest(BaseModel):
414     filename: str
415     total_size_bytes: Optional[int] = None
416
417
418 class UploadCompleteRequest(BaseModel):
419     total_parts: int
420
421
422 def _upload_cfg():
423     sec = getattr(global_config, "security", None)
424     upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427     exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
428     None) or ["mp4", "mov"])]
429     return upload_dir, max_bytes, part_bytes, exts
430
431
432 def _abort_upload(upload_id: str) -> None:
433     with _uploads_lock:
434         st = _uploads.pop(upload_id, None)
435         if st:
436             try:
437                 os.remove(st["tmp_path"])
438             except OSError:

```

```

438         pass
439
440
441     @app.post("/api/upload/init")
442     def upload_init(req: UploadInitRequest):
443         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
444         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
445         try:
446             name = safe_output_filename(req.filename)
447         except ValueError as e:
448             raise HTTPException(status_code=400, detail=str(e)) from e
449         ext = os.path.splitext(name)[1].lower().lstrip(".")
450         if ext not in exts:
451             raise HTTPException(status_code=400,
452                                 detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
453         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
454             raise HTTPException(status_code=413,
455                                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
456         os.makedirs(upload_dir, exist_ok=True)
457         upload_id = uuid.uuid4().hex
458         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
459         open(tmp_path, "wb").close()
460         with _uploads_lock:
461             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463                 "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466     @app.put("/api/upload/{upload_id}/part/{index}")
467     async def upload_part(upload_id: str, index: int, request: Request):
468         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469         _, max_bytes, part_bytes, _ = _upload_cfg()
470         with _uploads_lock:
471             st = _uploads.get(upload_id)
472             if st is None:
473                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474             if index != st["parts"]:
475                 raise HTTPException(status_code=409,
476                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478             received = 0
479             overflow = None
480             with open(st["tmp_path"], "ab") as fh:
481                 async for chunk in request.stream():
482                     received += len(chunk)
483                     if received > part_bytes:
484                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                         break
486                     if st["bytes"] + received > max_bytes:
487                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                         break
489                     fh.write(chunk)
490             if overflow:
491                 _abort_upload(upload_id)
492                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494             with _uploads_lock:
495                 st["parts"] += 1

```

```

496         st["bytes"] += received
497     return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499
500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
504         with _uploads_lock:
505             st = _uploads.get(upload_id)
506             if st is None:
507                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508             if st["parts"] == 0 or st["parts"] != req.total_parts:
509                 raise HTTPException(status_code=409,
510                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511             final_path = os.path.join(upload_dir, st["filename"])
512             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
513                 stem, ext = os.path.splitext(st["filename"])
514                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515             os.replace(st["tmp_path"], final_path)
516             with _uploads_lock:
517                 _uploads.pop(upload_id, None)
518             video_id = compute_video_id(final_path)
519             return {"path": os.path.abspath(final_path), "video_id": video_id,
520                   "size_bytes": os.path.getsize(final_path),
521                   "next": "POST /api/process with this path"}
522
523
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529             if not known:
530                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531             _abort_upload(upload_id)
532             return {"status": "aborted", "upload_id": upload_id}
533
534
535     # --- Highlight download (B3, PR-2) -----
536
537     @app.get("/api/highlights/{video_id}/{filename}")
538     def download_highlight(video_id: str, filename: str):
539         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
540         (which only writes into output_dir; the browser previously got back an unreachable
541         server-local path)."""
542         try:
543             name = safe_output_filename(filename)
544             except ValueError as e:
545                 raise HTTPException(status_code=400, detail=str(e)) from e
546             if not db_instance.get_video(video_id):
547                 raise HTTPException(status_code=404, detail="Indexed video record not found.")
548             output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
549             path = os.path.join(output_dir, name)
550             try:
551                 path = resolve_under_root(path, output_dir)
552             except ValueError as e:
553                 raise HTTPException(status_code=400, detail=str(e)) from e
554             if not os.path.isfile(path):

```

```

555         raise HTTPException(status_code=404,
556                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
557     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
558     return FileResponse(path, media_type=media, filename=name)
559
560
561 @app.post("/api/query")
562 def query_play_segments(req: QueryRequest):
563     filters = {
564         "server": req.server,
565         "receiver": req.receiver,
566         "duration_min": req.duration_min,
567         "event_type": req.event_type
568     }
569     segments = db_instance.query_play_segments(req.video_id, filters)
570     return {"play_segments": segments}
571
572 @app.post("/api/compile")
573 def compile_highlights(req: CompileRequest):
574     video = db_instance.get_video(req.video_id)
575     if not video:
576         raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578     # Retrieve matching play segments from db
579     all_segments = db_instance.query_play_segments(req.video_id, {})
580     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582     if not matched_segments:
583         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
584
585     # Reject path traversal / absolute output names (os.path.join would otherwise let an
586     # absolute or '..'-laden output_filename write anywhere on disk).
587     try:
588         out_name = safe_output_filename(req.output_filename)
589     except ValueError as e:
590         raise HTTPException(status_code=400, detail=str(e)) from e
591
592     # The configured shot-count floor (stitching.min_shot_count) applies on the API
593     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
594     # metadata are exempt: the floor filters known counts only, so explicitly
595     # requested manual/legacy segments can't silently vanish under the default floor.
596     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
597     kept = []
598     for s in matched_segments:
599         meta = s.get("metadata") or {}
600         sc = meta.get("shot_count") if isinstance(meta, dict) else None
601         try:
602             if sc is not None and int(sc) < stitching.min_shot_count:
603                 continue
604         except (TypeError, ValueError):
605             pass # unparseable count -> treat as unknown, keep
606         kept.append(s)
607     if len(kept) < len(matched_segments):
608         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
609                   f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
610     if not kept:
611         raise HTTPException(status_code=400,
612                             detail=f"All {len(matched_segments)} matching segments fall
below "
613

```

```

f"stitching.min_shot_count={stitching.min_shot_count}.")
614
615     # Extract start/end time pairs
616     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
617
618     # Run compiler with the configured stitching paddings + optional branding watermark
619     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
620     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
621                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
622     try:
623         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
624         return {"status": "success", "filepath": out_path}
625     except Exception as e:
626         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
628     # --- CLI Debug Utility & Orchestration ---
629     def run_cli_diagnose_clip(video_path: str, timestamp: float):
630         """CLI debugging utility to examine segment properties around a timestamp."""
631         print("-" * 60)
632         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
633         print("-" * 60)
634
635         cfg = load_config() # legacy wrapper still works, returns dict for now
636
637         # 1. Validation check diagnostics
638         print("[DIAGNOSTIC] Running validation framework...")
639         results = registry.run_all(video_path, cfg)
640         for r in results:
641             status_symbol = "[PASS]" if r.passed else "[FAIL]"
642             print(f" {status_symbol} {r.validator_name}: {r.message}")
643             if r.details:
644                 print(f"      Details: {r.details}")
645
646         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647         print("-" * 60)
648         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649         import cv2
650         cap = cv2.VideoCapture(video_path)
651         if not cap.isOpened():
652             print("[FAIL] OpenCV could not open video.")
653             return
654
655         fps = cap.get(cv2.CAP_PROP_FPS)
656         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658         start_frame = max(0, int((timestamp - 3.0) * fps))
659         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661         # Measure frame-by-frame changes in sample region
662         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663         prev_gray = None
664         motions = []
665
666         for _ in range(start_frame, end_frame):
667             ret, frame = cap.read()
668             if not ret:
669                 break
670             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671             gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673             if prev_gray is not None:
674                 diff = cv2.absdiff(prev_gray, gray)

```

```

675         _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676         motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677         motions.append(motion_ratio)
678         prev_gray = gray
679
680     cap.release()
681
682     if motions:
683         avg_motion = sum(motions) / len(motions)
684         # Support both legacy dict (from wrapper) and future direct model
685         if hasattr(cfg, "indexing"):
686             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687         else:
688             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
689         print(f" Sample Frame Count: {len(motions)}")
690         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
691         if avg_motion > threshold:
692             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693         else:
694             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
695     else:
696         print(" [ERROR] No visual frames were extracted in the sample range.")
697
698     print("=" * 60)
699
700     def main():
701         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
702         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
703         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
704         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
705         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
706         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
707         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
708         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
709
710         available_segmenters = list(segmenter_registry.list_available().keys())
711         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
712         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
713         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
714         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
715
716         args = parser.parse_args()
717
718         if args.setup:
719             # Invoke the diagnostics/bootstrap script (renamed from the old root
720             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
721             # build system). Resolve its path from this file so `indexer --setup`
722             # works regardless of the caller's cwd.
723             import subprocess

```

```

724         doctor_path = os.path.join(
725             os.path.dirname(os.path.abspath(__file__)),
726             "scripts", "doctor.py",
727         )
728         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
729         subprocess.run([sys.executable, doctor_path])
730         return
731
732     if args.trim_start is not None and args.trim_end is not None:
733         if not args.video_path:
734             print("[ERROR] Please provide --video-path to extract a segment.")
735             return
736
737         # Determine output path
738         out_filename = args.trim_output or "trimmed_segment.mp4"
739         if os.path.isabs(out_filename):
740             out_path = out_filename
741             out_dir = os.path.dirname(out_path)
742             out_name = os.path.basename(out_path)
743         else:
744             out_dir = args.output_dir
745             out_path = os.path.join(out_dir, out_filename)
746             out_name = out_filename
747
748         os.makedirs(out_dir, exist_ok=True)
749         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751         # global_config isn't initialized this early; load the typed config so the trim
752         # clip carries the same configured stitching paddings + watermark as a real
753         # highlight (watermark default-on).
754         stitching = load_app_config().stitching
755         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                 end_padding=stitching.end_padding,
757                                 watermark=stitching.watermark)
758         try:
759             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
760             args.trim_end)], out_name)
761             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
762         except Exception as e:
763             print(f"[ERROR] Failed to compile trimmed segment: {e}")
764             return
765
766     if args.debug_clip is not None:
767         if not args.video_path:
768             print("[ERROR] Please provide --video-path to debug a specific clip.")
769             return
770         run_cli_diagnose_clip(args.video_path, args.debug_clip)
771         return
772
773     # Initialize Global Config and DB
774     global global_config, db_instance
775     global_config = load_app_config() # returns typed AppConfig
776
777     # The CLI --output-dir overrides the configured output directory. It now lives
778     # on the typed model (OutputConfig.output_dir), so every consumer ? including
779     # /api/compile ? reads the same value instead of a write-only runtime hack.
780     global_config.output.output_dir = args.output_dir
781     os.makedirs(global_config.output.output_dir, exist_ok=True)
782
783     db_path = os.path.join(global_config.output.output_dir,
784                             global_config.output.db_name)
785     db_instance = Database(db_path)
786
787     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?

```

```

785     # the executor is in-process. Mark them failed so pollers see a terminal state.
786     swept = db_instance.fail_stale_jobs()
787     if swept:
788         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790     # CLI processing mode (no web UI needed)
791     if args.video_path and not args.server:
792         print(f"[CLI] Processing video file: {args.video_path}")
793         if not os.path.exists(args.video_path):
794             print(f"[FAIL] Video file not found: {args.video_path}")
795             return
796
797         video_id = compute_video_id(args.video_path)
798         was_reingest = db_instance.get_video(video_id) is not None
799
800         # Validate (with-context variant surfaces ffprobe_meta for the report)
801         print("[CLI] Validating...")
802         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803         failures = [r for r in val_results if not r.passed]
804
805         # Save video status
806         status = "failed" if failures else "processed"
807         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810         segmenter_actual = None
811         segmentation_ran = False
812         try:
813             print("\n" + "="*40)
814             print("VALIDATION SUMMARY")
815             print("="*40)
816             for r in val_results:
817                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
818                 print(f"{status_symbol} {r.validator_name}: {r.message}")
819             print("="*40 + "\n")
820
821             if failures:
822                 print("[FAIL] Video failed checks. Indexing halted.")
823                 return
824
825             # Index
826             segmenter_cls = segmenter_registry.get(seg_name)
827             if not segmenter_cls:
828                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
829                 return
830
831             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832             segmenter_actual = seg_name
833             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834             segmenter = segmenter_cls(db_instance, global_config)
835             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836             segmentation_ran = True
837
838             if isinstance(result, Failure):
839                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840                 print("[FAIL] Indexing halted for video.")
841                 segments = []
842             else:
843                 segments = result.value if hasattr(result, "value") else result
844                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play

```

```

segments inside database: {db_path}")
845         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846         _finalize_reingest(video_id, segments, play_wm, cand_wm)
847         finally:
848             # Always emit the per-video observability report. Never raises.
849             paths = emit_indexer_report(
850                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851                 val_results=val_results, val_context=val_context,
852                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854             )
855             if paths:
856                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
859             return
860
861         # Start Local Server Mode
862         if args.server or not args.video_path:
863             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
865             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
866             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
867             _get_executor().submit(_drain_due_jobs)
868             uvicorn.run(app, host="127.0.0.1", port=args.port)
869
870     if __name__ == "__main__":
871         main()
872

```

4. user (2026-06-18T07:47:53.489Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28

```

```

29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     @staticmethod
45     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
46         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
47
48         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
49         transaction, so an interrupted v3/v4 block can leave the column added while
50         ``user_version`` stays un-bumped ? and the next open would re-run the same
51         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
52         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
53         ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
54         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
55         """
56         try:
57             cursor.execute(ddl)
58         except sqlite3.OperationalError as exc:
59             if "duplicate column name" not in str(exc).lower():
60                 raise
61
62     def _init_db(self):
63         """Initializes tables for videos, play_segments, and events."""
64         with self._connect() as conn:
65             cursor = conn.cursor()
66
67             # Videos Table
68             cursor.execute("""
69                 CREATE TABLE IF NOT EXISTS videos (
70                     id TEXT PRIMARY KEY,
71                     filepath TEXT NOT NULL,
72                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73                     status TEXT NOT NULL,
74                     validation_results TEXT
75                 )
76             """)
77
78             # Play Segments Table (Extensible metadata JSON)
79             cursor.execute("""
80                 CREATE TABLE IF NOT EXISTS play_segments (
81                     id INTEGER PRIMARY KEY AUTOINCREMENT,
82                     video_id TEXT NOT NULL,
83                     start_time REAL NOT NULL,
84                     end_time REAL NOT NULL,
85                     duration REAL NOT NULL,
86                     server TEXT,
87                     receiver TEXT,
88                     metadata TEXT,
89                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
90                 )

```

```

91         """)
92
93     # Events Table
94     cursor.execute("""
95         CREATE TABLE IF NOT EXISTS events (
96             id INTEGER PRIMARY KEY AUTOINCREMENT,
97             segment_id INTEGER NOT NULL,
98             timestamp REAL NOT NULL,
99             type TEXT NOT NULL,
100             player TEXT,
101             details TEXT,
102             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
103         )
104     """)
105
106     # Versioned migrations live here. PRAGMA user_version is the schema
107     # contract ? bump it for every change and add an ordered `if version < N`
108     # block below. Tests assert the expected version, so accidental drift fails
109     CI.
110     version = cursor.execute("PRAGMA user_version").fetchone()[0]
111
112     if version < 1:
113         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
114         # Common fields are columns; detector-specific metrics go in `stats`
115         JSON,
116         # so a new segmenter reuses this table by setting its own
117         `detector`/`stats`.
118         cursor.execute("""
119             CREATE TABLE IF NOT EXISTS candidate_segments (
120                 id INTEGER PRIMARY KEY AUTOINCREMENT,
121                 video_id TEXT NOT NULL,
122                 detector TEXT NOT NULL,
123                 chunk_index INTEGER,
124                 start_time REAL NOT NULL,
125                 end_time REAL NOT NULL,
126                 duration REAL NOT NULL,
127                 confidence REAL,
128                 stats TEXT,
129                 detection_params TEXT,
130                 confirmed_segment_id INTEGER,
131                 human_label TEXT,
132                 notes TEXT,
133                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
134                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
135                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
136             )
137         """)
138         cursor.execute("""
139             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
ON candidate_segments(video_id, detector)
140         """)
141         cursor.execute("PRAGMA user_version = 1")
142
143     if version < 2:
144         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
145         submitted
146         # /api/process request. `status` is the EXECUTION state of the job
147         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
148         # that failed validation) lives inside `result` JSON as
149         result["status"].
150         # `result` is the full sync-era response dict (incl. report paths), so
151         the
152         # observability report is the job's artifact, per the plan.

```

```

148         cursor.execute("""
149             CREATE TABLE IF NOT EXISTS jobs (
150                 id TEXT PRIMARY KEY,
151                 video_path TEXT NOT NULL,
152                 video_id TEXT,
153                 segmenter TEXT,
154                 player_pool TEXT,
155                 max_frames INTEGER,
156                 status TEXT NOT NULL DEFAULT 'queued',
157                 progress TEXT,
158                 error TEXT,
159                 result TEXT,
160                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
161                 started_at TIMESTAMP,
162                 finished_at TIMESTAMP
163             )
164         """)
165         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
166         cursor.execute("PRAGMA user_version = 2")
167
168         if version < 3:
169             # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
170             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
171             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
172             # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
173             # the coordinator. ALTER (not recreate) so existing v2 job rows survive;
the
174             # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-
run them.
175             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy
TEXT")
176             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN
next_poll_at TIMESTAMP")
177             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
178             cursor.execute("PRAGMA user_version = 3")
179
180         if version < 4:
181             # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
182             # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
183             # path can scope rows to an owner. **NULL = local install = byte-
identical to
184             # today** ? every existing row stays NULL and every existing query
(which never
185             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
186             # the ADD COLUMNS are applied idempotently so an interrupted upgrade can
re-run them.
187             # Additive ONLY: no served-behavior change rides on this slice.
188             self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN
user_id TEXT")
189             self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD
COLUMN user_id TEXT")
190             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
191             # of index churn while making the eventual per-user lookups cheap.
192             cursor.execute(
193                 "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "

```

```

194         "WHERE user_id IS NOT NULL")
195     cursor.execute(
196         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
197         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
198     cursor.execute("PRAGMA user_version = 4")
199
200     if version < 5:
201         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
202         # (user_id,
203         # version): the resolved per-user `fusion_config` overlay + an INLINE
204         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
205         # and provenance that earned it. `is_active` pins the one row the
206         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
207         # only persists/loads it ? NOTHING reads it on the served path yet.
208         #
209         # user_id          ? owner of the model (NULL allowed = the local
210         # version          ? monotonic per-user model version
211         # baseline_version ? the shared baseline this was fit against
212         # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
213         # fusion_config     ? JSON per-user FusionConfig overlay (cue_flags
214         # classifier_json   ? JSON LogisticRegression.to_dict() (None =
215         # promotion_evidence ? JSON per_user_gate PromotionDecision (why it
216         # n_videos_at_fit   ? held-out-user corpus size when fit (min_n
217         # is_active         ? the one resolved row for this user (partial-
218         # unique below)
219         cursor.execute("""
220             CREATE TABLE IF NOT EXISTS user_models (
221                 id INTEGER PRIMARY KEY AUTOINCREMENT,
222                 user_id TEXT,
223                 version INTEGER NOT NULL DEFAULT 1,
224                 baseline_version TEXT,
225                 feature_schema_hash TEXT,
226                 fusion_config TEXT,
227                 classifier_json TEXT,
228                 promotion_evidence TEXT,
229                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
230                 is_active INTEGER NOT NULL DEFAULT 0,
231                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
232             )
233         """)
234         # One model version per user (NULLs compare distinct in SQLite, so the
235         # local
236         # install can still carry multiple versioned rows; the active-row guard
237         # below
238         # is what enforces a single served model).
239         cursor.execute("""
240             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
241             ON user_models(user_id, version)
242         """)
243         # At most ONE active model per user: a partial-unique index over the
244         # active rows.
245         # (SQLite treats NULL user_id rows as distinct, so the local install's
246         # single
247         # active row is still guarded ? there is one local user.)
248         cursor.execute("""

```

```

243             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
244             ON user_models(user_id) WHERE is_active = 1
245         """)
246         cursor.execute("PRAGMA user_version = 5")
247         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
248
249         conn.commit()
250
251     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
252         """Register or update a video row WITHOUT touching its child segments.
253
254         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
255         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
256         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
257         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
258         row in place; clearing segments is now an explicit, deliberate operation
259         (clear_video_data / prune_segments).
260         """
261         try:
262             with self._connect() as conn:
263                 conn.cursor().execute(
264                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
265                     "ON CONFLICT(id) DO UPDATE SET "
266                     "filepath=excluded.filepath, status=excluded.status, "
267                     "validation_results=excluded.validation_results",
268                     (video_id, filepath, status, json.dumps(validation_results))
269                 )
270                 conn.commit()
271             return True
272         except Exception as e:
273             logger.error(f"Failed to add video: {e}")
274             return False
275
276     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
277         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
278
279         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
280         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
281         """
282         with self._connect() as conn:
283             cur = conn.cursor()
284             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
285                             (video_id,)).fetchone()[0]
286             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
287                             (video_id,)).fetchone()[0]
288             return int(p), int(c)
289
290     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
291                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
292                       cand_after: Optional[int] = None) -> None:
293         """Delete play/candidate segments by id range (events cascade from
play_segments).
294
295         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
296         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial

```

```

rows
297         and keep the OLD ones when the run failed/produced nothing (`*_after``).
298         """
299         with self._connect() as conn:
300             cur = conn.cursor()
301             if play_upto is not None:
302                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
303             if play_after is not None:
304                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
305             if cand_upto is not None:
306                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
307             if cand_after is not None:
308                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
309             conn.commit()
310
311         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
312             try:
313                 with self._connect() as conn:
314                     cursor = conn.cursor()
315                     cursor.execute(
316                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
317                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
318                     )
319                     conn.commit()
320                     return cursor.lastrowid
321             except Exception as e:
322                 logger.error(f"Failed to add play segment: {e}")
323                 return None
324
325         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
326             try:
327                 with self._connect() as conn:
328                     conn.cursor().execute(
329                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
330                         (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
331                     )
332                     conn.commit()
333                     return True
334             except Exception as e:
335                 logger.error(f"Failed to add event: {e}")
336                 return False
337
338         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
339                                 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
340                                 stats: Optional[Dict[str, Any]] = None,
341                                 detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
342             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
343             detector-specific dicts stored as JSON. Returns the new candidate id, or
344             None."""
345             try:
346                 with self._connect() as conn:

```

```

346         cursor = conn.cursor()
347         cursor.execute(
348             """INSERT INTO candidate_segments
349                 (video_id, detector, chunk_index, start_time, end_time, duration,
350                  confidence, stats, detection_params)
351                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
352             (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
353             confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
354         )
355         conn.commit()
356         return cursor.lastrowid
357     except Exception as e:
358         logger.error(f"Failed to add candidate segment: {e}")
359         return None
360
361     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
362         """Record that a candidate detection was confirmed as a given play_segment."""
363         try:
364             with self._connect() as conn:
365                 conn.cursor().execute(
366                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
367                     (segment_id, candidate_id)
368                 )
369                 conn.commit()
370             return True
371         except Exception as e:
372             logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
373             return False
374
375     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
376                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
377         """Fetch candidate detections for the annotation / fine-tuning workflow.
378         `stats` and `detection_params` are deserialized from JSON."""
379         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
380         params: List[Any] = [video_id]
381
382         if detector:
383             query += " AND detector = ?"
384             params.append(detector)
385         if only_unlabeled:
386             query += " AND human_label IS NULL"
387
388         query += " ORDER BY start_time"
389
390         candidates = []
391         with self._connect() as conn:
392             rows = conn.cursor().execute(query, params).fetchall()
393             for row in rows:
394                 d = dict(row)
395                 d["stats"] = json.loads(d["stats"] or "{}")
396                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
397                 candidates.append(d)
398         return candidates
399
400     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
401         with self._connect() as conn:
402             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
403             if row:
404                 d = dict(row)
405                 d["validation_results"] = json.loads(d["validation_results"] or "[]")

```

```

406         return d
407     return None
408
409     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
410         """
411         Dynamically queries play segments based on filters:
412         - duration_min: float
413         - server: string
414         - receiver: string
415         - event_type: string (e.g. smash, foul)
416         """
417         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
418         params = [video_id]
419
420         if filters.get("duration_min"):
421             query += " AND r.duration >= ?"
422             params.append(filters["duration_min"])
423
424         if filters.get("server"):
425             query += " AND r.server = ?"
426             params.append(filters["server"])
427
428         if filters.get("receiver"):
429             query += " AND r.receiver = ?"
430             params.append(filters["receiver"])
431
432         if filters.get("event_type"):
433             # Check if there is an event of a specific type inside this segment
434             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
435             params.append(filters["event_type"])
436
437         segments_list = []
438         with self._connect() as conn:
439             cursor = conn.cursor()
440             rows = cursor.execute(query, params).fetchall()
441
442             for row in rows:
443                 r_dict = dict(row)
444                 r_id = r_dict["id"]
445                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
446
447                 # Fetch associated events
448                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
449                 r_dict["events"] = []
450                 for er in event_rows:
451                     e_dict = dict(er)
452                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
453                     r_dict["events"].append(e_dict)
454
455                 segments_list.append(r_dict)
456
457         return segments_list
458
459     def clear_video_data(self, video_id: str):
460         with self._connect() as conn:
461             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
462             conn.commit()
463
464     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
465     # Writers follow the repo convention (swallow + logger.error, return bool); the
466     # worker logs loudly on a failed transition so a job can't silently wedge.
467

```

```

468     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
469                     player_pool: Optional[List[str]] = None,
470                     max_frames: Optional[int] = None,
471                     policy: Optional[Dict[str, Any]] = None) -> bool:
472         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
473         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
474         try:
475             with self._connect() as conn:
476                 conn.cursor().execute(
477                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
478                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
479                     (job_id, video_path, segmenter,
480                     json.dumps(player_pool) if player_pool is not None else None,
481                     max_frames, json.dumps(policy) if policy is not None else None)
482                 )
483                 conn.commit()
484             return True
485         except Exception as e:
486             logger.error(f"Failed to create job {job_id}: {e}")
487             return False
488
489     def mark_job_running(self, job_id: str) -> bool:
490         try:
491             with self._connect() as conn:
492                 conn.cursor().execute(
493                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
494                     "progress='starting' WHERE id = ?", (job_id,))
495                 conn.commit()
496             return True
497         except Exception as e:
498             logger.error(f"Failed to mark job {job_id} running: {e}")
499             return False
500
501     def set_job_progress(self, job_id: str, progress: str) -> bool:
502         try:
503             with self._connect() as conn:
504                 conn.cursor().execute(
505                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
506                 conn.commit()
507             return True
508         except Exception as e:
509             logger.error(f"Failed to set progress for job {job_id}: {e}")
510             return False
511
512     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
513         try:
514             with self._connect() as conn:
515                 conn.cursor().execute(
516                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
517                 conn.commit()
518             return True
519         except Exception as e:
520             logger.error(f"Failed to set video_id for job {job_id}: {e}")
521             return False
522
523     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
524         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
525         validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
526         try:
527             with self._connect() as conn:
528                 conn.cursor().execute(
529                     "UPDATE jobs SET status='done', progress='finished', result=?, "

```

```

530         "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
531         (json.dumps(result), job_id))
532     conn.commit()
533     return True
534 except Exception as e:
535     logger.error(f"Failed to mark job {job_id} done: {e}")
536     return False
537
538 def mark_job_failed(self, job_id: str, error: str) -> bool:
539     """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
540     try:
541         with self._connect() as conn:
542             conn.cursor().execute(
543                 "UPDATE jobs SET status='failed', error=?, "
544                 "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
545             conn.commit()
546         return True
547     except Exception as e:
548         logger.error(f"Failed to mark job {job_id} failed: {e}")
549         return False
550
551 def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
552     """Fetch one job; `result`/`player_pool` JSON deserialized."""
553     with self._connect() as conn:
554         row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
555 (job_id,)).fetchone()
556         if not row:
557             return None
558         d = dict(row)
559         d["result"] = json.loads(d["result"]) if d["result"] else None
560         d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
561 None
562         d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
563         return d
564
565 # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
566 -----
567 def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
568     """Attach/replace a job's JSON ExecutionPolicy."""
569     try:
570         with self._connect() as conn:
571             conn.cursor().execute(
572                 "UPDATE jobs SET policy=? WHERE id = ?",
573                 (json.dumps(policy) if policy is not None else None, job_id))
574             conn.commit()
575         return True
576     except Exception as e:
577         logger.error(f"Failed to set policy for job {job_id}: {e}")
578         return False
579
580 def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
581 None) -> bool:
582     """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
583 releasing the
584 worker. `claim_due_job` re-picks it when due ? cooperative, no master.
585 ``next_poll_at`` is
586 computed in SQLite's own UTC format so it compares directly to
587 CURRENT_TIMESTAMP."""
588     try:
589         with self._connect() as conn:
590             conn.cursor().execute(
591                 "UPDATE jobs SET status='waiting', "
592                 "next_poll_at=datetime('now', ?), "
593                 "progress=COALESCE(?, progress) WHERE id = ?",
594                 (f"+{max(0, int(delay_sec))} seconds", progress, job_id))

```

```

588         conn.commit()
589         return True
590     except Exception as e:
591         logger.error(f"Failed to set job {job_id} waiting: {e}")
592         return False
593
594     def seconds_until_next_due(self) -> Optional[float]:
595         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
596         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
597         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
598         try:
599             with self._connect() as conn:
600                 row = conn.cursor().execute(
601                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
602                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
603                     "FROM jobs WHERE status='waiting']").fetchone()
604                 if row is None or row["secs"] is None:
605                     return None
606                 return max(0.0, float(row["secs"]))
607         except Exception as e:
608             logger.error(f"Failed to compute next-due delay: {e}")
609             return None
610
611     def claim_due_job(self) -> Optional[Dict[str, Any]]:
612         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
613         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
614         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
615         first. Returns the claimed job dict, or None if nothing is runnable."""
616         try:
617             with self._connect() as conn:
618                 cur = conn.cursor()
619                 row = cur.execute(
620                     "SELECT id, status FROM jobs WHERE status='queued' "
621                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
622                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
623                 if not row:
624                     return None
625                 cur.execute(
626                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
627                     "WHERE id=? AND status=?", (row["id"], row["status"]))
628                 conn.commit()
629                 if cur.rowcount != 1:
630                     return None # lost the race to a concurrent
worker
631                 return self.get_job(row["id"])
632         except Exception as e:
633             logger.error(f"Failed to claim a due job: {e}")
634             return None
635
636     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
637         """Crash recovery: any job still queued/running from a previous process is dead
638         (the executor is in-process). Mark failed so clients see a terminal state, not a
639         hang. Called once at startup. Returns the number of jobs swept."""
640         try:
641             with self._connect() as conn:
642                 cur = conn.cursor()

```

```

643         cur.execute(
644             "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
645             "WHERE status IN ('queued', 'running')", (reason,))
646         conn.commit()
647         return cur.rowcount
648     except Exception as e:
649         logger.error(f"Failed to sweep stale jobs: {e}")
650         return 0
651
652     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
653     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
654     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
655     # serving.py) resolves them, and wiring that bridge into the production decision
seam
656     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
657     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
658     # JSON-serialised dicts; the rest are scalar columns.
659
660     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
661                           baseline_version: Optional[str] = None,
662                           feature_schema_hash: Optional[str] = None,
663                           fusion_config: Optional[Dict[str, Any]] = None,
664                           classifier_json: Optional[Dict[str, Any]] = None,
665                           promotion_evidence: Optional[Dict[str, Any]] = None,
666                           n_videos_at_fit: int = 0,
667                           is_active: bool = False) -> Optional[int]:
668         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
669
670         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
671         rowid). When `is_active` is True this row becomes the user's single active
model ?
672         any previously-active row for the SAME user is first cleared, so the
673         `idx_user_models_one_active` partial-unique invariant always holds (one active
model
674         per user). Returns the row id, or None on failure.
675         """
676         try:
677             with self._connect() as conn:
678                 cur = conn.cursor()
679                 if is_active:
680                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
681                     if user_id is None:
682                         cur.execute(
683                             "UPDATE user_models SET is_active = 0 "
684                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
685                             (version,))
686                     else:
687                         cur.execute(
688                             "UPDATE user_models SET is_active = 0 "
689                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
690                             (user_id, version))
691                 cur.execute(
692                     """INSERT INTO user_models
693                     (user_id, version, baseline_version, feature_schema_hash,
694                     fusion_config, classifier_json, promotion_evidence,
695                     n_videos_at_fit, is_active)
696                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
697                     ON CONFLICT(user_id, version) DO UPDATE SET
698                     baseline_version=excluded.baseline_version,
699                     feature_schema_hash=excluded.feature_schema_hash,
700                     fusion_config=excluded.fusion_config,
701                     classifier_json=excluded.classifier_json,

```

```

702             promotion_evidence=excluded.promotion_evidence,
703             n_videos_at_fit=excluded.n_videos_at_fit,
704             is_active=excluded.is_active""",
705             (user_id, version, baseline_version, feature_schema_hash,
706             json.dumps(fusion_config) if fusion_config is not None else None,
707             json.dumps(classifier_json) if classifier_json is not None else
None,
708             json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
709             n_videos_at_fit, 1 if is_active else 0))
710         conn.commit()
711         rid = cur.execute(
712             "SELECT id FROM user_models WHERE "
713             + ("user_id IS NULL" if user_id is None else "user_id = ?")
714             + " AND version = ?",
715             ((version,) if user_id is None else (user_id, version))).fetchone()
716         return int(rid[0]) if rid else None
717     except Exception as e:
718         logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
719         return None
720
721     def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
722         """Return the single active per-user model row for ``user_id`` (NULL = local
install),
723         with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
724         or None if the user has no active model. Does not touch the served pipeline."""
725         with self._connect() as conn:
726             if user_id is None:
727                 row = conn.cursor().execute(
728                     "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
729             else:
730                 row = conn.cursor().execute(
731                     "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
732                     (user_id,)).fetchone()
733             if not row:
734                 return None
735             d = dict(row)
736             for col in ("fusion_config", "classifier_json", "promotion_evidence"):
737                 d[col] = json.loads(d[col]) if d[col] else None
738             return d
739

```

5. assistant (2026-06-18T07:47:55.603Z)

Let me examine the remaining files in the target areas.

6. user (2026-06-18T07:47:57.070Z)

```

1         """Canonical file-identity hashing for the indexer.
2
3         `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4         (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5         `scripts/run_process_and_stitch.py`)
6         defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7         The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8         any future change to one copy would silently diverge the DB key.
9
10        This module is the single source of truth. Import `compute_video_id` everywhere a
11        video needs to be keyed.
12        """
13        from __future__ import annotations

```

```

14
15     import hashlib
16
17     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18     # was centralized. The chunk size does not affect the resulting digest ? it only
19     # bounds memory use while streaming large files.
20     _CHUNK_SIZE = 65536
21
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                 hasher.update(chunk)
33         return hasher.hexdigest()
34

```

7. user (2026-06-18T07:47:57.254Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2     import os
3     from typing import Optional
4
5
6     def safe_output_filename(name: str) -> str:
7         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10        and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11        primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12        """
13        if not name or not name.strip():
14            raise ValueError("output_filename must not be empty")
15        if "\x00" in name:
16            raise ValueError("output_filename must not contain a null byte")
17        if "/" in name or "\\" in name:
18            raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19        if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20            raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21        return name
22
23
24     def resolve_under_root(path: str, root: str) -> str:
25         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26         rp = os.path.realpath(path)
27         rr = os.path.realpath(root)
28         try:
29             contained = os.path.commonpath([rp, rr]) == rr
30         except ValueError: # different drives on Windows
31             contained = False
32         if not contained:
33             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34         return rp
35
36
37     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:

```

```

38         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40         With no allowed_root (default, single-user local mode) only the null-byte check
41         applies,
42         so the local 'point it at any of my videos' workflow is preserved. Setting an
43         allowed_root (a future hosted/tenant deployment) enforces containment.
44         """
45         if "\x00" in path:
46             raise ValueError("video_path must not contain a null byte")
47         if allowed_root:
48             resolve_under_root(path, allowed_root)

```

8. assistant (2026-06-18T07:47:59.774Z)

Now let me look at the config and providers directories.

9. user (2026-06-18T07:48:00.078Z)

```

backend\config\__init__.py
backend\config\loader.py
backend\config\models.py

```

10. user (2026-06-18T07:48:00.303Z)

```

backend\providers\base.py
backend\providers\__init__.py
backend\providers\gemini.py

```

11. user (2026-06-18T07:48:04.477Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
""") -> list[str]:
26         """Dotted paths in `data` that the typed config will not honor.
27
28         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31         user's intent is lost ? collect every such key so load_config can warn.
32         """

```

```

33     unknown: list[str] = []
34     fields = model_cls.model_fields
35     for key, val in data.items():
36         if key not in fields:
37             unknown.append(prefix + key)
38             continue
39         ann = fields[key].annotation
40         if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41             unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42     return unknown
43
44
45 def _get_built_in_defaults() -> dict[str, Any]:
46     """Return a plain dict of the current built-in defaults.
47
48     The AppConfig model is the single source of truth for defaults.
49     """
50     return AppConfig().model_dump(mode="json")
51
52
53 def load_config(path: str | Path | None = None) -> AppConfig:
54     """Load configuration with the following precedence:
55
56     1. Built-in defaults defined in the Pydantic models.
57     2. Values from the JSON file (if present and readable).
58     3. (Future) environment / CLI overrides.
59
60     Missing file or unreadable file ? returns a fully defaulted AppConfig.
61     Unknown keys in the file are tolerated (extra="allow" on the model).
62     """
63     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65     file_data: dict[str, Any] = {}
66     if cfg_path.exists():
67         try:
68             with open(cfg_path, "r", encoding="utf-8") as f:
69                 file_data = json.load(f)
70         except Exception as exc:
71             # Non-fatal: continue with defaults + whatever we could parse.
72             # In production we might log here.
73             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75     if file_data:
76         unknown = _unknown_key_paths(file_data, AppConfig)
77         if unknown:
78             logger.warning(
79                 "[CONFIG WARNING] %s contains keys the typed config does not "
80                 "recognize (typo'd or removed knobs ? top-level extras are kept "
81                 "but unread; unknown section keys are silently dropped): %s",
82                 cfg_path, ", ".join(sorted(unknown)),
83             )
84
85     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86     merged = {_get_built_in_defaults(), **file_data}
87
88     return AppConfig.model_validate(merged)
89
90
91 def save_default_config(path: str | Path | None = None) -> Path:
92     """Write a fresh default config.json.
93
94     Used by scripts/doctor.py diagnostics and the --setup CLI path.
95     Overwrites if the file exists.

```

```

96     """
97     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98     cfg = AppConfig()
99     cfg_path.parent.mkdir(parents=True, exist_ok=True)
100    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101    return cfg_path
102
103
104    # Backwards-compatible alias used by some older call sites during transition.
105    def get_default_config() -> dict[str, Any]:
106        """Return defaults as a plain dict (for transitional use only)."""
107        return _get_builtin_defaults()
108

```

12. user (2026-06-18T07:48:04.841Z)

```

1    """Pydantic models for the application configuration.
2
3    This module defines the canonical, validated, typed configuration for the
4    Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6    The models support graceful migration from old config.json files via extra="allow".
7    """
8
9    from __future__ import annotations
10
11    from typing import Any, List, Literal, Optional
12
13    from pydantic import BaseModel, Field, field_validator, model_validator
14
15    Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18    class ValidationRelevanceConfig(BaseModel):
19        court_green_lower: list[int] = Field(default=[35, 40, 40])
20        court_green_upper: list[int] = Field(default=[85, 255, 255])
21        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24    class ValidationConfig(BaseModel):
25        min_resolution_width: int = 1280
26        min_resolution_height: int = 720
27        min_fps: float = 29.9
28        min_blur_threshold: float = 20.0
29        max_scene_cuts_per_minute: float = 0.5
30        relevance: ValidationRelevanceConfig =
31    Field(default_factory=ValidationRelevanceConfig)
32
33    class TrackNetConfig(BaseModel):
34        wsl_distro: str = "Ubuntu"
35        repo_dir: str = "~/models/TrackNetV4"
36        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37        conda_env: str = "TrackNetV4"
38        weights_path: str = ""
39        queue_length: int = 5
40        stage_in_wsl: bool = True
41        wsl_stage_dir: str = "~/clips"
42        timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43        velocity_threshold: float = 0.02
44        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45        # decoded frame cache) are deleted automatically ? they are pure intermediates,
46        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48        keep_frames: bool = False

```

```

49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76         # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77         # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78         # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79         # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80         # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81         stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         """
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101         # enabled ? so the default path never imports torch / touches the GPU.
102         cue_flags: dict[str, bool] = Field(default_factory=dict)

```

```

103         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105         min_crossings: int = Field(default=0, ge=0)
106         # Served Gate B: when the person cue is on, drop windows with median in-court
players
107         # < this. 0 = OFF.
108         min_players: int = Field(default=0, ge=0)
109         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110         court_polygon: Optional[list[list[float]]] = None
111         # Net line for the person two-sided-motion split (person features only ? the shuttle
112         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113         net_axis: Literal["x", "y"] = "y"
114         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
133         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
134         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139         max_window_duration: float = 6.0
140         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
141         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
142         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144         # Only cuts (recall can't drop). OFF by default until measured/promoted.
145         window_hard_cap: bool = False
146         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
147         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148         # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.

```

```

150      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
152      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153      window_inactivity_end: float = 1.5
154      inactivity_rally_thresh: float = 0.20
155      inactivity_min_density: float = 0.30
156      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
158      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
161      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164      window_blob_fallback: bool = False
165      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169      window_blob_factor: float = 4.0
170      log_yolo_candidates: bool = True
171      store_yolo_candidates: bool = True
172      ai_handoff_padding: float = 2.0
173      ai_max_workers: int = 5
174      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
175      # chunks of a high-bitrate (4K) source blow past the provider upload cap
176      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
177      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
178      # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
179      # Matches gemini_refine's --clip-scale-width default.
180      ai_chunk_scale_width: int = 1280
181      # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
182      # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
183      # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
184      # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
185      # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
186      skip_ai_handoff: bool = False
187      # Optional debug knob: trim every video to the first N seconds before processing.
188      debug_duration: Optional[float] = None
189
190      tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
191      wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
192      fusion: FusionConfig = Field(default_factory=FusionConfig)
193
194      @field_validator("default_segmenter")
195      @classmethod
196      def segmenter_must_be_registered(cls, v: str) -> str:

```

```

197         # Registry check happens at runtime in main/segmenter selection.
198         # Here we just allow any string for forward compatibility.
199         return v
200
201     @model_validator(mode="after")
202     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
203         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
204         # step makes `while t < duration: t += step` loop forever, growing memory before
any
205         # GPU work. Reject the bad config at load time instead.
206         if self.chunk_overlap >= self.chunk_duration:
207             raise ValueError(
208                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
209                 f"({self.chunk_duration}); otherwise chunking never advances."
210             )
211         return self
212
213
214     class OutputConfig(BaseModel):
215         default_highlights_name: str = "highlights.mp4"
216         db_name: str = "sports_indexer.db"
217         # Directory where the DB, highlight reels, and processed clips are written.
218         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
219         output_dir: str = "output"
220
221
222     class WatermarkConfig(BaseModel):
223         """Branding overlay burned into each highlight clip during the per-clip re-encode
224         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
225         (under `stitching.watermark` in config.json) to turn it off. The text is fully
226         configurable. The concat stage stays stream-copy (-c copy)."""
227         enabled: bool = True
228         text: str = "Generated by Khelsutra.guru"
229         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
230         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
231         font: str = "Sans" # fontconfig family used when font_path is empty
232         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
233         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
234         box: bool = True # faint backing box behind the text for legibility
235         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
236         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
237
238
239     class StitchingConfig(BaseModel):
240         begin_padding: float = 0.5
241         end_padding: float = 0.5
242         use_cache: bool = False
243         min_shot_count: int = 1
244         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
245
246
247     class LoggingConfig(BaseModel):
248         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
249         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
250         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
251         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
252         # them to WARNING; set true to restore full chatter for request-flow debugging.
253         verbose_http: bool = False
254
255

```

```

256 class SecurityConfig(BaseModel):
257     # When set, /api/process video_path must resolve under this directory (hosted/tenant
258     # mode). Default None preserves the single-user local 'any local file' workflow.
259     allowed_video_root: Optional[str] = None
260
261     # --- Chunked upload (B2, PR-2) ---
262     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
263     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
264     # contain upload_dir or /api/process will reject the uploaded paths.
265     upload_dir: str = "output/uploads"
266     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
267     # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
268     max_upload_mb: int = 4096
269     max_part_mb: int = 95
270     allowed_upload_extensions: List[str] = ["mp4", "mov"]
271
272
273 class AppConfig(BaseModel):
274     """Root configuration object.
275
276     All runtime configuration should be accessed via an instance of this class
277     (or the loader) rather than raw dict lookups.
278     """
279
280     sport: Sport = "badminton"
281     ai_provider: str = "gemini"
282     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
283     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
284     ai_model: str = "gemini-flash-latest"
285
286     validation: ValidationConfig = Field(default_factory=ValidationConfig)
287     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
288     output: OutputConfig = Field(default_factory=OutputConfig)
289     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
290     security: SecurityConfig = Field(default_factory=SecurityConfig)
291     logging: LoggingConfig = Field(default_factory=LoggingConfig)
292
293     # Allow extra keys during transition so existing config.json files do not break.
294     model_config = {
295         "extra": "allow",
296         "validate_assignment": True,
297     }
298
299     # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
300     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
301     # attribute access / the typed config. A nested *section* is returned as a plain
dict
302     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
303     # section model instead breaks it (a model has no .get), which bit every validator
on
304     # real runs. Scalar leaves are returned as-is.
305     def get(self, key: str, default: Any = None) -> Any:
306         _MISSING = object()
307         val: Any = _MISSING
308         if key in type(self).model_fields or hasattr(self, key):
309             val = getattr(self, key, _MISSING)
310         if val is _MISSING:
311             # Legacy convenience: find a leaf key inside any nested section. Sections
are
312             # discovered dynamically from the model fields, so a future top-level
section
313             # (e.g. `audio`, `jobs`) is covered automatically without editing a

```

```

hardcoded list.
314         for fname in type(self).model_fields:
315             section = getattr(self, fname, None)
316             if isinstance(section, BaseModel) and hasattr(section, key):
317                 val = getattr(section, key)
318                 break
319             if val is _MISSING:
320                 return default
321             if isinstance(val, BaseModel):
322                 return val.model_dump(mode="json")
323             return val
324
325     def __getitem__(self, key: str) -> Any:
326         val = self.get(key)
327         if val is None and key not in type(self).model_fields:
328             # Re-raise like a real dict for unknown top-level
329             raise KeyError(key)
330         return val
331
332
333     # Convenience type alias for callers that want to annotate
334     Config = AppConfig
335

```

13. assistant (2026-06-18T07:48:07.057Z)

Now let me look at the providers.

14. user (2026-06-18T07:48:08.004Z)

```

1     import os
2     import time
3     import json
4     import logging
5     from typing import Tuple
6     from google import genai
7     from google.genai import types
8     from backend.providers.base import AIVideoProvider
9     from backend.infrastructure.execution_policy import (
10         ExecutionPolicy, PolicyTimeout, poll_until, run_bounded,
11     )
12
13     logger = logging.getLogger(__name__)
14
15     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
16     # each uploaded clip is below this. Oversize uploads are refused (not silently
17     # attempted).
18     MAX_UPLOAD_MB = 500
19
20     class GeminiProvider(AIVideoProvider):
21         def __init__(self, api_key: str, model_name: str):
22             super().__init__(api_key, model_name)
23             # Initialize client lazily to avoid connection/auth issues during class
24             # instantiation
25             self._client = None
26
27         @property
28         def client(self):
29             if self._client is None:
30                 self._client = genai.Client(api_key=self.api_key)
31             return self._client
32
33         def _safe_delete(self, video_file) -> None:
34             """Best-effort cleanup of an uploaded File (never raises)."""
35             try:

```

```

34         self.client.files.delete(name=video_file.name)
35     except Exception:
36         pass
37
38     def analyze_video(self, video_path: str, prompt: str,
39                       chunk_duration: float = 0.0,
40                       label: str = "") -> Tuple[list, dict]:
41         log_prefix = f"Gemini {label}" if label else "Gemini"
42
43         metrics = {
44             "upload_time": 0.0,
45             "process_time": 0.0,
46             "gen_time": 0.0
47         }
48
49         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
50         try:
51             size_mb = os.path.getsize(video_path) / (1024 * 1024)
52         except OSError:
53             size_mb = 0.0
54         if size_mb > MAX_UPLOAD_MB:
55             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? ")
56             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
57             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
58             metrics["oversize_skipped"] = True # callers aggregate this into the run
report
59             return [], metrics
60
61         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
62         logger.info(f"[{log_prefix}] Uploading {video_path}...")
63         t_upload_start = time.time()
64         video_file = None
65         for attempt in range(3):
66             try:
67                 video_file = self.client.files.upload(file=video_path)
68                 metrics["upload_time"] = time.time() - t_upload_start
69                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
70                 break
71             except Exception as e:
72                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
73                 if attempt < 2:
74                     time.sleep(2 * (attempt + 1))
75         if video_file is None:
76             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
77             return [], metrics
78
79         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
80         # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
81         t_process_start = time.time()
82         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
83
84         def _processed():
85             nonlocal video_file
86             video_file = self.client.files.get(name=video_file.name)
87             return None if video_file.state.name == "PROCESSING" else video_file
88

```

```

89         try:
90             video_file = poll_until(_processed, process_policy,
91                                     label=f"[log_prefix] processing", logger=logger)
92         except PolicyTimeout:
93             logger.error(f"[log_prefix] Processing exceeded
94 {process_policy.max_wait_sec:.0f}s ? giving up.")
95             self._safe_delete(video_file)
96             return [], metrics
97         except Exception as e:
98             logger.error(f"[log_prefix] Status check failed: {e}")
99             self._safe_delete(video_file)
100            return [], metrics
101        if video_file.state.name == "FAILED":
102            logger.error(f"[log_prefix] Video processing failed:
103 {video_file.error.message}")
104            self._safe_delete(video_file)
105            return [], metrics
106
107        metrics["process_time"] = time.time() - t_process_start
108        logger.info(f"[log_prefix] Processing finished in
109 {metrics['process_time']:.1f}s.")
110
111        # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
112        # killer that was
113        # missing: a generate() that never returns previously blocked forever) + retry
114        # the 503/network
115        # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
116        # attempts).
117        logger.info(f"[log_prefix] Submitting multimodal request to
118 {self.model_name}...")
119        t_gen_start = time.time()
120        gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
121        backoff_base_sec=3.0)
122        raw_text = None
123
124        def _generate():
125            return self.client.models.generate_content(
126                model=self.model_name,
127                contents=[video_file, prompt],
128                config=types.GenerateContentConfig(
129                    response_mime_type="application/json", temperature=0.0),
130            )
131
132        try:
133            response = run_bounded(_generate, gen_policy, label=f"[log_prefix]
134 generate", logger=logger)
135            raw_text = response.text
136            metrics["gen_time"] = time.time() - t_gen_start
137            logger.info(f"[log_prefix] Response received in
138 {metrics['gen_time']:.1f}s:\n{raw_text}")
139        except Exception as e:
140            # PolicyTimeout (hang) or the last 503 ?
141            both end here
142            logger.error(f"[log_prefix] generate gave up after retries: {e}")
143            self._safe_delete(video_file)
144
145        if not raw_text:
146            return [], metrics
147
148        # 4. Parse JSON
149        try:
150            segments = json.loads(raw_text)
151            if not isinstance(segments, list):
152                logger.error(f"[log_prefix] Response is not a JSON array.")
153            return [], metrics
154        return segments, metrics

```

```

143         except Exception as e:
144             logger.error(f"[{log_prefix}] JSON parse failed: {e}")
145             return [], metrics
146

```

15. user (2026-06-18T07:48:08.589Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Tuple
3
4     class AIVideoProvider(ABC):
5         def __init__(self, api_key: str, model_name: str):
6             self.api_key = api_key
7             self.model_name = model_name
8
9         @abstractmethod
10        def analyze_video(self, video_path: str, prompt: str,
11                          chunk_duration: float = 0.0,
12                          label: str = "") -> Tuple[list, dict]:
13            """Sends a video file to the AI model along with a prompt, and returns a tuple
14            of
15            (segments_list, metrics_dict).
16
17            Args:
18                video_path: Path to the video file to analyze.
19                prompt: Prompt instructions.
20                chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
21                label: Diagnostic label for logging.
22
23            Returns:
24                Tuple containing:
25                - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
26                  ...}])
27                - A dictionary of performance metrics (e.g. {"upload_time": ...,
28                  "process_time": ..., "gen_time": ...})
29            """
30            pass
31

```

16. assistant (2026-06-18T07:48:13.541Z)

Now I have a strong picture. Let me dig into a few potential issues. First, the `claim\_due\_job` flow vs `mark\_job\_running`. In `claim\_due\_job`, the job is set to `status='running'` but `started\_at=CURRENT\_TIMESTAMP`. Note that `mark\_job\_running` is defined but let me check if it's actually used anywhere - that's not a bug per se. Let me focus on real reachable defects.

Let me check the `upload\_part` concurrency issue and the `st["bytes"]` accounting. Let me look more carefully at potential issues in the upload flow and the `\_uploads\_lock` usage.

Let me examine the providers `\_\_init\_\_` and how the provider is constructed/retried, and check the execution_policy.

17. user (2026-06-18T07:48:14.939Z)

```

1     from typing import Dict, Type
2     from backend.providers.base import AIVideoProvider
3     from backend.providers.gemini import GeminiProvider
4
5     class ProviderRegistry:
6         def __init__(self):
7             self._registry: Dict[str, Type[AIVideoProvider]] = {}
8             # Register Gemini provider
9             self.register("gemini", GeminiProvider)
10

```

```

11         def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12             self._registry[name] = provider_cls
13
14         def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15             if name not in self._registry:
16                 raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17             return self._registry[name](api_key=api_key, model_name=model_name)
18
19         def list_available(self) -> Dict[str, str]:
20             return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22     provider_registry = ProviderRegistry()
23

```

18. user (2026-06-18T07:48:15.429Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    """
17    from __future__ import annotations
18
19    import logging
20    import threading
21    import time as _time
22    from dataclasses import asdict, dataclass
23    from enum import Enum
24    from typing import Callable, Optional, TypeVar
25
26    T = TypeVar("T")
27
28    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29    LOG = logging.getLogger("execution.policy")
30
31
32    class WaitMode(str, Enum):
33        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34        WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35        ABORT = "abort" # one attempt; on hang/fail, stop immediately
36        BLOCK = "block" # bounded in-process blocking wait (no
reschedule)

```

```

37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
41         ``max_wait_sec``."""
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
46         EXECUTION_POLICY.md)."""
47         mode: WaitMode = WaitMode.WAIT_AND_RESUME
48         poll_every_n_sec: float = 10.0
49         op_timeout_sec: float = 120.0          # HARD per-attempt deadline ? the hang-killer
50         max_wait_sec: float = 1800.0          # total wall budget for a poll loop
51         max_retries: int = 5                   # retries (=> max_retries+1 attempts) for
52         backoff_base_sec: float = 3.0          # exponential: backoff_base * 2**attempt
53         on_timeout: str = "reschedule"         # reschedule | partial | fail (consumed by
54         resumable: bool = True
55
56         def to_dict(self) -> dict:
57             d = asdict(self)
58             d["mode"] = self.mode.value
59             return d
60
61         @classmethod
62         def from_dict(cls, d: dict) -> "ExecutionPolicy":
63             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
64             kw = {k: v for k, v in d.items() if k in known}
65             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
66                 kw["mode"] = WaitMode(kw["mode"])
67             return cls(**kw)
68
69     #: The sane default (also what a job gets if it declares no policy).
70     DEFAULT_POLICY = ExecutionPolicy()
71
72
73     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75         return policy.backoff_base_sec * (2 ** attempt)
76
77
78     def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79                    label: str = "op", logger: Optional[logging.Logger] = None,
80                    sleep: Callable[[float], None] = _time.sleep) -> T:
81         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
82         backoff.
83
84         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
85         hung, or re-raises the
86         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
87         retries).
88
89         """
90         log = logger or LOG
91         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
92         last_exc: Optional[BaseException] = None
93         for attempt in range(attempts):
94             box: dict = {}
95             done = threading.Event()
96
97             # box/done bound as defaults (not closed over) ? the thread is started + joined

```

```

within this
94         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95         def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96             try:
97                 _box["val"] = fn()
98             except BaseException as e:         # relay ANY error (incl. timeouts) to the
caller thread
99                 _box["exc"] = e
100             finally:
101                 _done.set()
102
103             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
104             if done.wait(timeout=policy.op_timeout_sec):
105                 if "exc" in box:
106                     last_exc = box["exc"]
107                     log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108                 else:
109                     if attempt:
110                         log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111                         return box["val"]         # type: ignore[return-value]
112                 else:
113                     last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114                     log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                                 policy.op_timeout_sec)
116                     if attempt < attempts - 1:
117                         wait = _backoff_sec(policy, attempt)
118                         log.debug("%s: backing off %.1fs before retry", label, wait)
119                         sleep(wait)
120                     assert last_exc is not None
121                     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122                     raise last_exc
123
124
125         def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                       label: str = "op", logger: Optional[logging.Logger] = None,
127                       sleep: Callable[[float], None] = _time.sleep,
128                       monotonic: Callable[[], float] = _time.monotonic) -> T:
129             """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130             ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132             Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133             """
134             log = logger or LOG
135             start = monotonic()
136             polls = 0
137             log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                     label, policy.poll_every_n_sec, policy.max_wait_sec)
139             while True:
140                 result = check()
141                 if result is not None:
142                     log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                     return result
144                 elapsed = monotonic() - start
145                 if elapsed >= policy.max_wait_sec:
146                     log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,

```

```

elapsed)
147             raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

19. assistant (2026-06-18T07:48:18.732Z)

Now let me focus on the `upload\_part` concurrency/accounting carefully. There's a subtle issue I want to verify with the test file. Let me check the existing tests for uploads and jobs to understand intended behavior.

20. user (2026-06-18T07:48:19.298Z)

tests\test_upload_download.py

21. user (2026-06-18T07:48:19.640Z)

tests\test_async_jobs.py
tests\test_job_self_scheduling.py
tests\test_job_worker_loop.py

22. user (2026-06-18T07:48:19.662Z)

tests\test_database.py

23. user (2026-06-18T07:48:23.174Z)

```

1      """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3      Contract under test:
4      - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5      - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
(413)
6      - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7      - DELETE /api/upload/{id} ? abort removes partial state
8      - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9      """
10     import os
11
12     import pytest
13
14     pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
15     from fastapi.testclient import TestClient
16
17     import backend.main as m
18     from backend.infrastructure.database import Database
19     from backend.config.models import AppConfig
20     from backend.utils.hashing import compute_video_id
21
22
23     @pytest.fixture
24     def env(tmp_path, monkeypatch):
25         db = Database(str(tmp_path / "t.db"))
26         cfg = AppConfig()
27         cfg.output.output_dir = str(tmp_path / "out")
28         cfg.security.upload_dir = str(tmp_path / "up")
29         os.makedirs(cfg.output.output_dir, exist_ok=True)
30         monkeypatch.setattr(m, "db_instance", db)
31         monkeypatch.setattr(m, "global_config", cfg)
32         monkeypatch.setattr(m, "_uploads", {}) # isolate upload state per test
33         client = TestClient(m.app)

```

```

34         return client, db, tmp_path, cfg
35
36
37     def _init(client, filename="match.mp4", total=None):
38         body = {"filename": filename}
39         if total is not None:
40             body["total_size_bytes"] = total
41         return client.post("/api/upload/init", json=body)
42
43
44     # --- init -----
45
46     def test_init_rejects_bad_extension(env):
47         client = env[0]
48         r = _init(client, "malware.exe")
49         assert r.status_code == 400
50         assert "not allowed" in r.json()["detail"]
51
52
53     def test_init_rejects_traversal_filename(env):
54         client = env[0]
55         assert _init(client, "../evil.mp4").status_code == 400
56         assert _init(client, "a/b.mp4").status_code == 400
57
58
59     def test_init_rejects_oversize_declared_total(env):
60         client, db, tmp_path, cfg = env
61         cfg.security.max_upload_mb = 1
62         r = _init(client, total=2 * 1024 * 1024)
63         assert r.status_code == 413
64
65
66     # --- full lifecycle -----
67
68     def test_roundtrip_assembles_parts_and_matches_md5(env):
69         client, db, tmp_path, cfg = env
70         up = _init(client).json()
71         parts = [b"a" * 1000, b"b" * 2000, b"c" * 500]
72         for i, blob in enumerate(parts):
73             r = client.put(f"/api/upload/{up['upload_id']}/part/{i}", content=blob)
74             assert r.status_code == 200
75             assert r.json()["received_parts"] == i + 1
76         done = client.post(f"/api/upload/{up['upload_id']}/complete", json={"total_parts":
37})
77         assert done.status_code == 200
78         body = done.json()
79         assert body["size_bytes"] == 3500
80         with open(body["path"], "rb") as fh:
81             assert fh.read() == b"".join(parts)
82         assert body["video_id"] == compute_video_id(body["path"])
83         # partial file cleaned up, registry entry gone
84         assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
85         assert client.put(f"/api/upload/{up['upload_id']}/part/3", content=b"x").status_code
== 404
86
87
88     def test_same_filename_twice_does_not_overwrite(env):
89         client = env[0]
90         paths = []
91         for content in (b"first", b"second"):
92             up = _init(client).json()
93             client.put(f"/api/upload/{up['upload_id']}/part/0", content=content)
94             done = client.post(f"/api/upload/{up['upload_id']}/complete",
json={"total_parts": 1})

```

```

95         paths.append(done.json()["path"])
96     assert paths[0] != paths[1]
97     with open(paths[0], "rb") as fh:
98         assert fh.read() == b"first" # first upload untouched
99     with open(paths[1], "rb") as fh:
100         assert fh.read() == b"second"
101
102
103     # --- sequencing & caps -----
104
105     def test_out_of_order_part_is_409(env):
106         client = env[0]
107         up = _init(client).json()
108         assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 409
109
110
111     def test_unknown_upload_id_is_404(env):
112         client = env[0]
113         assert client.put("/api/upload/nope/part/0", content=b"x").status_code == 404
114         assert client.post("/api/upload/nope/complete", json={"total_parts": 1}).status_code
== 404
115         assert client.delete("/api/upload/nope").status_code == 404
116
117
118     def test_complete_with_wrong_part_count_is_409(env):
119         client = env[0]
120         up = _init(client).json()
121         client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
122         assert client.post(
123             f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 2}).status_code
== 409
124         # zero-part complete is also a 409
125         up2 = _init(client).json()
126         assert client.post(
127             f"/api/upload/{up2['upload_id']}/complete", json={"total_parts": 0}).status_code
== 409
128
129
130     def test_oversize_part_aborts_upload(env):
131         client, db, tmp_path, cfg = env
132         cfg.security.max_part_mb = 1
133         up = _init(client).json()
134         r = client.put(f"/api/upload/{up['upload_id']}/part/0",
135                       content=b"x" * (1024 * 1024 + 10))
136         assert r.status_code == 413
137         # aborted: id unknown afterwards, no partials on disk
138         assert client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x").status_code
== 404
139         assert not [f for f in os.listdir(cfg.security.upload_dir) if
140 f.startswith(".partial-")]
141
142
143     def test_total_cap_across_parts_aborts_upload(env):
144         client, db, tmp_path, cfg = env
145         cfg.security.max_upload_mb = 1
146         up = _init(client).json()
147         ok = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x" * (700 *
148 1024))
149         assert ok.status_code == 200
150         r = client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x" * (700 * 1024))
151         assert r.status_code == 413
152
153     def test_abort_endpoint_removes_state(env):

```

```

153         client, db, tmp_path, cfg = env
154         up = _init(client).json()
155         client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
156         assert client.delete(f"/api/upload/{up['upload_id']}").status_code == 200
157         assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
158         == 404
159         assert not [f for f in os.listdir(cfg.security.upload_dir) if
160         f.startswith(".partial-")]
161
162     # --- download (B3)
163     -----
164
165     def test_download_unknown_video_404(env):
166         client = env[0]
167         assert client.get("/api/highlights/nope/reel.mp4").status_code == 404
168
169     def test_download_traversal_filename_rejected(env):
170         client, db, tmp_path, cfg = env
171         db.add_video("vid", "/x.mp4", "processed", [])
172         r = client.get("/api/highlights/vid/..%2Fevil.mp4")
173         assert r.status_code in (400, 404) # 400 from safe-name; 404 if the router
174         normalizes
175         r2 = client.get("/api/highlights/vid/a%5Cb.mp4") # backslash separator
176         assert r2.status_code == 400
177
178     def test_download_missing_file_404(env):
179         client, db, tmp_path, cfg = env
180         db.add_video("vid", "/x.mp4", "processed", [])
181         assert client.get("/api/highlights/vid/reel.mp4").status_code == 404
182
183     def test_download_streams_compiled_file(env):
184         client, db, tmp_path, cfg = env
185         db.add_video("vid", "/x.mp4", "processed", [])
186         blob = b"\x00\x00fake-mp4-bytes" * 100
187         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
188             fh.write(blob)
189         r = client.get("/api/highlights/vid/reel.mp4")
190         assert r.status_code == 200
191         assert r.content == blob
192         assert r.headers["content-type"].startswith("video/mp4")
193         assert "reel.mp4" in r.headers.get("content-disposition", "")
194

```

24. assistant (2026-06-18T07:48:31.840Z)

Now let me examine the `upload\_part` total cap logic carefully. There's a real bug I want to verify. In `upload\_part`:

```

```python
received = 0
overflow = None
with open(st["tmp_path"], "ab") as fh:
 async for chunk in request.stream():
 received += len(chunk)
 if received > part_bytes:
 ...
 if st["bytes"] + received > max_bytes:
 ...
 fh.write(chunk)
...

```

The total cap check ``st["bytes"] + received > max_bytes`` is done BEFORE writing, but it counts cumulative ``received`` per part. The test ``test_total_cap_across_parts_aborts_upload`` passes. Let me think about the bug. Actually ``received`` already accumulates this part's bytes, so ``st["bytes"] + received`` is the running total. That's correct.

Let me look at a more interesting concern. In ``download_highlight``, ``safe_output_filename`` rejects path separators, but ``resolve_under_root`` is then applied. That's belt-and-suspenders. Fine.

Let me focus on ``compile_highlights`` ? the ``matched_segments`` filter ``s["id"] in req.segment_ids``. The ``segment_ids`` is ``List[int]``. Fine.

Now let me check the ``query_play_segments`` with ``duration_min``. ``if filters.get("duration_min")`` ? this fails when ``duration_min`` is ``0.0`` (falsy). But that's a minor semantic edge. Let me check a more critical area ? the migration ladder and the ``_finalize_reingest`` watermark logic for a concurrency concern. Let me check ``test_database.py`` for the migration version assertions and look at ``fail_stale_jobs``/``claim_due_job`` race.

## 25. user (2026-06-18T07:48:32.716Z)

```
1 import sqlite3
2 import pytest
3 from backend.infrastructure.database import Database
4
5 def test_database_init(tmp_path):
6 db_path = tmp_path / "test.db"
7 Database(str(db_path)) # instantiation creates the schema
8
9 conn = sqlite3.connect(str(db_path))
10 cursor = conn.cursor()
11 cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
12 tables = [r[0] for r in cursor.fetchall()]
13 assert "videos" in tables
14 assert "play_segments" in tables
15 conn.close()
16
17 def test_database_video(tmp_path):
18 db = Database(str(tmp_path / "test.db"))
19
20 vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
21 assert vid_added is True
22
23 video = db.get_video("vid1")
24 assert video["id"] == "vid1"
25 assert video["filepath"] == "vid1.mp4"
26 assert video["status"] == "pending"
27 assert video["validation_results"] == [{"val": 1}]
28
29 db.clear_video_data("vid1")
30 assert db.get_video("vid1") is None
31
32 def test_database_play_segment(tmp_path):
33 db = Database(str(tmp_path / "test.db"))
34 db.add_video("vid1", "vid1.mp4", "pending", [])
35
36 sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
37 assert sid is not None
38
39 db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
40
41 segments = db.query_play_segments("vid1", {"server": "PlayerA"})
42 assert len(segments) == 1
43 assert segments[0]["id"] == sid
44 assert segments[0]["start_time"] == 10.0
45 assert segments[0]["server"] == "PlayerA"
```

```

46 assert segments[0]["metadata"]["foo"] == "bar"
47
48 assert len(segments[0]["events"]) == 1
49 assert segments[0]["events"][0]["type"] == "smash"
50 assert segments[0]["events"][0]["details"]["speed"] == 300
51
52
53 # --- candidate_segments schema + behavior -----
54
55 # The schema contract: any future change to candidate_segments columns must
56 # consciously update this set, otherwise the test below fails (regression guard).
57 EXPECTED_CANDIDATE_COLUMNS = {
58 "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
59 "duration", "confidence", "stats", "detection_params",
60 "confirmed_segment_id", "human_label", "notes", "created_at",
61 "user_id", # v4: personalization groundwork (nullable; NULL = local install)
62 }
63
64
65 def test_candidate_schema_contract(tmp_path):
66 Database(str(tmp_path / "test.db")) # instantiation creates the schema
67 conn = sqlite3.connect(str(tmp_path / "test.db"))
68 cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69 conn.close()
70
71 names = {c[1] for c in cols}
72 assert names == EXPECTED_CANDIDATE_COLUMNS
73
74 # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
75 by_name = {c[1]: c for c in cols}
76 assert by_name["id"][5] == 1 # primary key
77 for col in ("video_id", "detector", "start_time", "end_time", "duration"):
78 assert by_name[col][3] == 1, f"{col} should be NOT NULL"
79
80
81 def test_schema_version_is_current(tmp_path):
82 db_path = tmp_path / "test.db"
83 Database(str(db_path))
84 conn = sqlite3.connect(str(db_path))
85 version = conn.execute("PRAGMA user_version").fetchone()[0]
86 conn.close()
87 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
88 assert version == 5
89
90
91 def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92 db_path = tmp_path / "legacy.db"
93 # Simulate a pre-v1 database that only has the old tables and no version.
94 conn = sqlite3.connect(str(db_path))
95 conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
96 conn.commit()
97 conn.close()
98
99 # Opening with the new Database should add candidate_segments without error...
100 Database(str(db_path))
101 # ...and running init again must be a no-op (idempotent).
102 Database(str(db_path))
103
104 conn = sqlite3.connect(str(db_path))
105 tables = [r[0] for r in conn.execute(
106 "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
107 version = conn.execute("PRAGMA user_version").fetchone()[0]
108 conn.close()

```

```

109 assert "candidate_segments" in tables
110 assert "jobs" in tables # v2 (async job model, #16)
111 assert "user_models" in tables # v5 (per-user model store)
112 assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
113
114
115 def test_candidate_round_trip(tmp_path):
116 db = Database(str(tmp_path / "test.db"))
117 db.add_video("vid1", "vid1.mp4", "processed", [])
118
119 cid = db.add_candidate_segment(
120 "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
121 stats={"peak_velocity": 0.8, "track_id_count": 2},
122 detection_params={"frame_skip": 3},
123)
124 assert cid is not None
125
126 cands = db.query_candidate_segments("vid1")
127 assert len(cands) == 1
128 c = cands[0]
129 assert c["detector"] == "yolo_hybrid"
130 assert c["start_time"] == 10.0
131 assert c["duration"] == 15.0
132 assert c["confidence"] == 0.42
133 assert c["stats"]["peak_velocity"] == 0.8
134 assert c["detection_params"]["frame_skip"] == 3
135 assert c["confirmed_segment_id"] is None
136
137 # detector filter
138 assert db.query_candidate_segments("vid1", detector="motion") == []
139 assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
140
141
142 def test_candidate_link_and_unlabeled_filter(tmp_path):
143 db = Database(str(tmp_path / "test.db"))
144 db.add_video("vid1", "vid1.mp4", "processed", [])
145
146 cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
147 sid = db.add_play_segment("vid1", 1.0, 4.0)
148
149 assert db.link_candidate_to_segment(cid, sid) is True
150 assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
151
152 # only_unlabeled returns it while human_label is NULL
153 assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
154 with db._get_connection() as conn:
155 conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
156 ("true_rally", cid))
157 conn.commit()
158 assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
159
160
161 def test_candidate_cascade_on_video_delete(tmp_path):
162 db = Database(str(tmp_path / "test.db"))
163 db.add_video("vid1", "vid1.mp4", "processed", [])
164 db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
165
166 db.clear_video_data("vid1")
167 assert db.query_candidate_segments("vid1") == []
168
169
170 # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
171
172 def _table_cols(db_path, table):
173 conn = sqlite3.connect(str(db_path))

```

```

174 cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
175 conn.close()
176 return cols
177
178
179 def test_v4_user_id_columns_present_and_nullable(tmp_path):
180 db_path = tmp_path / "test.db"
181 Database(str(db_path))
182 assert "user_id" in _table_cols(db_path, "videos")
183 assert "user_id" in _table_cols(db_path, "candidate_segments")
184
185 # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
186 conn = sqlite3.connect(str(db_path))
187 for table in ("videos", "candidate_segments"):
188 by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
189 assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
190 conn.close()
191
192
193 def test_v4_existing_writers_leave_user_id_null(tmp_path):
194 """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
195 row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
196 db_path = tmp_path / "test.db"
197 db = Database(str(db_path))
198 db.add_video("vid1", "vid1.mp4", "processed", [])
199 db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
200
201 conn = sqlite3.connect(str(db_path))
202 vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
203 cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
204 conn.close()
205 assert vu is None
206 assert cu is None
207 # Existing readers are unaffected (no user_id filter in the query path).
208 assert len(db.query_candidate_segments("vid1")) == 1
209
210
211 def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212 """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
213 columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
214 db_path = tmp_path / "legacy_v3.db"
215 db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
216 # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
217 db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
218 db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
219 conn = sqlite3.connect(str(db_path))
220 conn.execute("DROP INDEX IF EXISTS idx_videos_user")
221 conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
222 conn.execute("DROP TABLE IF EXISTS user_models")
223 # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
224 # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
225 conn.execute("ALTER TABLE videos DROP COLUMN user_id")
226 conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
227 conn.execute("PRAGMA user_version = 3")
228 conn.commit()
229 conn.close()

```

```

230 assert "user_id" not in _table_cols(db_path, "videos")
231
232 # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
233 db2 = Database(str(db_path))
234 conn = sqlite3.connect(str(db_path))
235 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
236 assert "user_models" in {r[0] for r in conn.execute(
237 "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
238 conn.close()
239 assert "user_id" in _table_cols(db_path, "videos")
240 assert "user_id" in _table_cols(db_path, "candidate_segments")
241
242 v = db2.get_video("vidA")
243 assert v is not None and v["validation_results"] == [{"v": 1}]
244 cands = db2.query_candidate_segments("vidA")
245 assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
246
247
248 def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
249 """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
250 on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
251 bump leaves the columns present while the version is stale ? re-opening re-runs the
252 bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
253 instead of raising ``OperationalError: duplicate column name``."""
254 db_path = tmp_path / "interrupted.db"
255 Database(str(db_path)) # fully migrate to the current version (v3/v4 columns all
present)
256
257 # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-
committed,
258 # but the process died before the version bump, so user_version is stuck at 2.
259 conn = sqlite3.connect(str(db_path))
260 conn.execute("DROP TABLE IF EXISTS user_models") # a true v2 DB lacks the
v5 table
261 conn.execute("PRAGMA user_version = 2")
262 conn.commit()
263 conn.close()
264
265 # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
266 # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
267 Database(str(db_path))
268 conn = sqlite3.connect(str(db_path))
269 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
270 conn.close()
271 # Columns/tables are all present after the clean re-apply.
272 assert "policy" in _table_cols(db_path, "jobs")
273 assert "user_id" in _table_cols(db_path, "videos")
274 assert "user_id" in _table_cols(db_path, "candidate_segments")
275 conn = sqlite3.connect(str(db_path))
276 assert "user_models" in {r[0] for r in conn.execute(
277 "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
278 conn.close()
279
280
281 def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
282 """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
283 OperationalError (e.g. a genuinely broken migration against a missing table) must
still
284 propagate ? the swallow is narrow, not a blanket except."""
285 db_path = tmp_path / "helper.db"
286 Database(str(db_path)) # current schema; jobs.policy already
exists
287 conn = sqlite3.connect(str(db_path))
288 cur = conn.cursor()
289 # Re-adding an existing column is tolerated (the crash-safety path).

```

```

290 Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
291 # An unrelated OperationalError (no such table) is NOT masked.
292 with pytest.raises(sqlite3.OperationalError):
293 Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
294 conn.close()
295
296
297 # --- v5: per-user model store CRUD -----
298
299 def test_user_models_schema_contract(tmp_path):
300 db_path = tmp_path / "test.db"
301 Database(str(db_path))
302 cols = _table_cols(db_path, "user_models")
303 assert cols == {
304 "id", "user_id", "version", "baseline_version", "feature_schema_hash",
305 "fusion_config", "classifier_json", "promotion_evidence",
306 "n_videos_at_fit", "is_active", "created_at",
307 }
308
309
310 def test_upsert_and_get_active_user_model_round_trip(tmp_path):
311 db = Database(str(tmp_path / "test.db"))
312 rid = db.upsert_user_model(
313 "user-1", version=1, baseline_version="base-2026-06",
314 feature_schema_hash="abc123",
315 fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
316 classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
317 promotion_evidence={"promote": True, "n": 6},
318 n_videos_at_fit=6, is_active=True,
319)
320 assert rid is not None
321
322 m = db.get_active_user_model("user-1")
323 assert m is not None
324 assert m["user_id"] == "user-1"
325 assert m["version"] == 1
326 assert m["baseline_version"] == "base-2026-06"
327 assert m["feature_schema_hash"] == "abc123"
328 assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
329 assert m["classifier_json"]["w"] == [0.1, 0.2]
330 assert m["promotion_evidence"]["promote"] is True
331 assert m["n_videos_at_fit"] == 6
332 assert m["is_active"] == 1
333
334
335 def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
336 db = Database(str(tmp_path / "test.db"))
337 assert db.get_active_user_model("nobody") is None
338
339 # An inactive model is not returned as active.
340 db.upsert_user_model("user-2", version=1, is_active=False)
341 assert db.get_active_user_model("user-2") is None
342
343
344 def test_upsert_is_idempotent_on_user_version(tmp_path):
345 db = Database(str(tmp_path / "test.db"))
346 rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
347 rid2 = db.upsert_user_model(
348 "user-3", version=1, n_videos_at_fit=7, is_active=True,
349 fusion_config={"min_crossings": 5},
350)
351 # Same (user, version) ? same row updated in place, not a duplicate.
352 assert rid1 == rid2
353 m = db.get_active_user_model("user-3")
354 assert m["n_videos_at_fit"] == 7

```

```

355 assert m["fusion_config"] == {"min_crossings": 5}
356
357 conn = sqlite3.connect(str(tmp_path / "test.db"))
358 count = conn.execute(
359 "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
360 conn.close()
361 assert count == 1
362
363
364 def test_activating_new_version_demotes_prior_active(tmp_path):
365 db = Database(str(tmp_path / "test.db"))
366 db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
367 db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)
368
369 active = db.get_active_user_model("user-4")
370 assert active["version"] == 2 # the newer model is the single active one
371
372 # Exactly one active row for the user (the partial-unique invariant holds).
373 conn = sqlite3.connect(str(tmp_path / "test.db"))
374 n_active = conn.execute(
375 "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
376 1").fetchone()[0]
377 conn.close()
378 assert n_active == 1
379
380 def test_null_user_id_local_install_round_trip(tmp_path):
381 """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
382 '=NULL')."""
383 db = Database(str(tmp_path / "test.db"))
384 db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
385 n_videos_at_fit=5, is_active=True)
386 m = db.get_active_user_model(None)
387 assert m is not None
388 assert m["user_id"] is None
389 assert m["fusion_config"] == {"min_crossings": 2}
390 # A named user does NOT see the local row.
391 assert db.get_active_user_model("user-x") is None
392
393 def test_user_model_json_columns_default_none(tmp_path):
394 """A config-only model (no classifier) stores NULL classifier_json and deserialises
395 to None."""
396 db = Database(str(tmp_path / "test.db"))
397 db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
398 n_videos_at_fit=5, is_active=True)
399 m = db.get_active_user_model("user-5")
400 assert m["classifier_json"] is None
401 assert m["promotion_evidence"] is None
402 assert m["fusion_config"] == {"min_crossings": 4}

```

## 26. assistant (2026-06-18T07:48:41.084Z)

Now let me verify a specific concern in `claim_due_job`. It claims a job and sets `status='running'`, but it never resets `next_poll_at` or `error`/`finished_at`. For a re-claimed `waiting` job that previously had an error... actually waiting jobs never had errors. Let me focus.

Let me examine the `get_job` deserialization edge with `mark_job_done` storing `result` as JSON. In `_run_claimed_job`, `db_instance.mark_job_done(job_id, result)` where `result` contains `play_segments` which are raw segment dicts. These come from the segmenter. The `result` dict is JSON-serialized via `json.dumps(result)`. If `play_segments` contains non-JSON-serializable objects, `mark_job_done` would fail (caught, logged, returns False, job stays 'running'). But that's an existing pattern.

Let me now focus on a concrete defect. Looking at `process\_video` (the submit endpoint) and `create\_job`: the job is created with `video\_id=NULL`. Then `\_drain\_due\_jobs` is submitted. Let me check `test\_async\_jobs.py` and `test\_job\_worker\_loop.py` to understand the wiring and whether `mark\_job\_running` being unused matters.

## 27. user (2026-06-18T07:48:42.088Z)

```
1 """Async job model (DEFERRED_HARDENING_PLAN #16).
2
3 Contract under test:
4 - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
5 - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
6 - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
7 verdict (e.g. failed validation) lives in result["status"].
8 - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
9 - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
10 - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
11 """
12 import sqlite3
13 import time
14 from concurrent.futures import ThreadPoolExecutor
15
16 import pytest
17
18 pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
19 from fastapi.testclient import TestClient
20
21 import backend.main as m
22 from backend.infrastructure.database import Database
23 from backend.config.models import AppConfig
24 from backend.pipeline.validators.base import ValidationResult
25
26
27 class _InlineExecutor:
28 """submit() runs the fn synchronously ? deterministic tests, no threads."""
29
30 def submit(self, fn, *args, **kwargs):
31 fn(*args, **kwargs)
32
33
34 @pytest.fixture
35 def env(tmp_path, monkeypatch):
36 db = Database(str(tmp_path / "t.db"))
37 cfg = AppConfig()
38 monkeypatch.setattr(m, "db_instance", db)
39 monkeypatch.setattr(m, "global_config", cfg)
40 monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
41 client = TestClient(m.app)
42 return client, db, tmp_path, monkeypatch
43
44
45 def _video(tmp_path, name="m.mp4"):
46 p = tmp_path / name
47 p.write_bytes(b"not-a-real-video-but-hashable")
48 return p
49
50
51 def _pass():
52 return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
53
54
55 def _fail():
56 return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]
```

```

57
58
59 class _FakeSeg:
60 def __init__(self, db, cfg):
61 self.db = db
62
63 def process_video(self, video_id, video_path, *a, **k):
64 sid = self.db.add_play_segment(video_id, 0.0, 5.0)
65 return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
66
67
68 class _RaisingSeg:
69 def __init__(self, db, cfg):
70 self.db = db
71
72 def process_video(self, video_id, video_path, *a, **k):
73 self.db.add_play_segment(video_id, 9.0, 10.0) # half-written row pre-crash
74 raise RuntimeError("boom")
75
76
77 # --- migration -----
78
79 def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
80 db = Database(str(tmp_path / "fresh.db"))
81 with db._get_connection() as conn:
82 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2); the jobs table arrived
at v2/v3.
83 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
84 cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
85 assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
cols
86 assert {"policy", "next_poll_at"} <= cols # v3 self-scheduling columns
87
88
89 def test_migration_upgrades_v1_db_in_place(tmp_path):
90 path = str(tmp_path / "v1.db")
91 conn = sqlite3.connect(path)
92 conn.execute(
93 "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
94 "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
95 # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
96 # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
97 conn.execute(
98 "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
99 "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
100 "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)")
101 conn.execute("PRAGMA user_version = 1")
102 conn.commit()
103 conn.close()
104
105 db = Database(path) # ladder should take it 1 ? 5
106 with db._get_connection() as c:
107 assert c.execute("PRAGMA user_version").fetchone()[0] == 5
108 assert c.execute(
109 "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs'").fetchone()
110 # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
111 # These were dropped in prior edit; add (do not replace) per review.
112 tables = {r[0] for r in c.execute(
113 "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
114 assert "user_models" in tables

```

```

115 cand_cols = {r["name"] for r in c.execute(
116 "PRAGMA table_info(candidate_segments)").fetchall()}
117 assert "user_id" in cand_cols
118
119
120 def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
121 """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
122 (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
123 _run_claimed_job)
124 produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
125 Failure, and empty-run branches. Real DB queries using vid (no self-compares or
126 tautologies).
127 """
128 client, db, tpath, mp = env
129 vid_file = _video(tpath)
130 vid = m.compute_video_id(str(vid_file))
131 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
132 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
133
134 req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")
135
136 # Direct CLI-style path
137 direct_out = m._execute_processing(req)
138 segs = db.query_play_segments(vid, {})
139 assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
140 assert direct_out.get("result", {}).get("status") in (None, "success", "processed")
141 or "play_segments" in str(direct_out)
142
143 # Job-worker path via public API submit (exercises job creation + inline executor +
144 internal _run_claimed_job)
145 rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
146 "x"})
147 job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
148 segs_j = db.query_play_segments(vid, {})
149 assert job_out["result"]["status"] == "success"
150 assert len(segs_j) >= 1
151
152 # Explicit job-worker via _run_claimed_job (create + mark running + drive the
153 claimed runner)
154 vid_file2 = _video(tpath, name="m2.mp4")
155 vid2 = m.compute_video_id(str(vid_file2))
156 jid = f"parity-{vid2[:8]}"
157 db.create_job(jid, str(vid_file2))
158 db.mark_job_running(jid)
159 job_row = db.get_job(jid)
160 m._run_claimed_job(job_row)
161 segs2 = db.query_play_segments(vid2, {})
162 assert len(segs2) >= 1, f"_run_claimed_job path must produce segments for vid
163 {vid2}"
164
165 # Failure branch (direct + job submit)
166 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
167 req_f = m.ProcessRequest(video_path=str(vid_file))
168 direct_f = m._execute_processing(req_f)
169 rf = client.post("/api/process", json={"video_path": str(vid_file)})
170 jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
171 assert direct_f.get("result", {}).get("status") == "failed" or
172 jobf["result"]["status"] == "failed"
173
174 # Empty-run branch (success verdict but zero play_segments from this execution)
175 class _EmptySeg:
176 def __init__(self, db, cfg):
177 self.db = db
178 def process_video(self, video_id, video_path, *a, **k):
179 return []

```

```

172 mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
173 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
174 req_e = m.ProcessRequest(video_path=str(vid_file))
175 de = m._execute_processing(req_e)
176 re = client.post("/api/process", json={"video_path": str(vid_file)})
177 je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
178 assert de.get("result", {}).get("status") in (None, "success")
179 assert je["result"]["status"] == "success"
180 # (reingest safety for prior on empty/fail covered in dedicated tests)
181
182
183 # --- submit-time fast-fail -----
184
185 def test_submit_returns_202_with_job_id(env):
186 client, db, tmp_path, mp = env
187 vid_file = _video(tmp_path)
188 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
189 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
190 r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
191 assert r.status_code == 202
192 body = r.json()
193 assert body["job_id"]
194 assert body["status"] == "queued"
195 assert body["poll"].endswith(body["job_id"])
196
197
198 def test_submit_rejects_null_byte_path(env):
199 client = env[0]
200 r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
201 assert r.status_code == 400
202
203
204
205 def test_submit_missing_file_is_404(env):
206 client, db, tmp_path, mp = env
207 r = client.post("/api/process", json={"video_path": str(tmp_path / "nope.mp4")})
208 assert r.status_code == 404
209
210
211 def test_submit_unknown_segmenter_400_and_no_job_row(env):
212 client, db, tmp_path, mp = env
213 vid_file = _video(tmp_path)
214 mp.setattr(m.segmenter_registry, "get", lambda name: None)
215 mp.setattr(m.segmenter_registry, "list_available", lambda: {"motion": "..."})
216 r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"nope"})
217 assert r.status_code == 400
218 assert "Available segmenters" in r.json()["detail"]
219 with db._get_connection() as c:
220 assert c.execute("SELECT COUNT(*) FROM jobs").fetchone()[0] == 0
221
222
223 # --- end-to-end job lifecycle (inline executor = deterministic) -----
224
225 def test_job_done_with_segments_and_video_id(env):
226 client, db, tmp_path, mp = env
227 vid_file = _video(tmp_path)
228 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
229 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
230
231 job_id = client.post(
232 "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
233 r = client.get(f"/api/jobs/{job_id}")

```

```

234 assert r.status_code == 200
235 body = r.json()
236 assert body["status"] == "done"
237 assert body["progress"] == "finished"
238 assert body["error"] is None
239 assert body["result"]["status"] == "success"
240 assert len(body["result"]["play_segments"]) == 1
241 assert "reports" in body # None without obsreport; key must exist per the contract
242
243 vid = m.compute_video_id(str(vid_file))
244 assert body["video_id"] == vid
245 assert len(db.query_play_segments(vid, {})) == 1
246 assert db.get_video(vid)["status"] == "processed"
247
248
249 def test_job_done_but_pipeline_verdict_failed_preserves_prior(env):
250 client, db, tmp_path, mp = env
251 vid_file = _video(tmp_path)
252 vid = m.compute_video_id(str(vid_file))
253 db.add_video(vid, str(vid_file), "processed", [])
254 db.add_play_segment(vid, 1.0, 2.0) # prior good result
255 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
256
257 job_id = client.post("/api/process", json={"video_path":
str(vid_file)}).json()["job_id"]
258 body = client.get(f"/api/jobs/{job_id}").json()
259 # Worker finished fine ? the PIPELINE verdict is the failure.
260 assert body["status"] == "done"
261 assert body["result"]["status"] == "failed"
262 # PR-C semantics survive the async move: prior segments intact.
263 assert len(db.query_play_segments(vid, {})) == 1
264
265
266 def test_worker_crash_marks_failed_and_restores_prior_segments(env):
267 client, db, tmp_path, mp = env
268 vid_file = _video(tmp_path)
269 vid = m.compute_video_id(str(vid_file))
270 db.add_video(vid, str(vid_file), "processed", [])
271 db.add_play_segment(vid, 1.0, 2.0) # prior good result
272 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
273 mp.setattr(m.segmenter_registry, "get", lambda name: _RaisingSeg)
274
275 job_id = client.post(
276 "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
277 body = client.get(f"/api/jobs/{job_id}").json()
278 assert body["status"] == "failed"
279 assert "RuntimeError" in body["error"]
280 assert body["result"] is None
281 # The half-written 9.0?10.0 row was pruned; the prior 1.0?2.0 row survives.
282 segs = db.query_play_segments(vid, {})
283 assert len(segs) == 1 and segs[0]["start_time"] == 1.0
284
285
286 def test_progress_stages_are_recorded(env):
287 client, db, tmp_path, mp = env
288 vid_file = _video(tmp_path)
289 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
290 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
291
292 stages = []
293 real = db.set_job_progress
294
295 def spy(job_id, progress):
296 stages.append(progress)

```

```

297 return real(job_id, progress)
298
299 mp.setattr(db, "set_job_progress", spy)
300 client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
301 assert "hashing" in stages
302 assert any(s.startswith("segmenting") for s in stages)
303
304
305 # --- polling endpoint -----
306
307 def test_get_unknown_job_is_404(env):
308 client = env[0]
309 assert client.get("/api/jobs/deadbeef").status_code == 404
310
311
312 # --- crash recovery -----
313
314 def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
315 client, db, tmp_path, mp = env
316 db.create_job("q1", "/v.mp4")
317 db.create_job("r1", "/v.mp4")
318 db.mark_job_running("r1")
319 db.create_job("d1", "/v.mp4")
320 db.mark_job_done("d1", {"status": "success"})
321
322 swept = db.fail_stale_jobs("interrupted by server restart")
323 assert swept == 2
324 assert db.get_job("q1")["status"] == "failed"
325 assert db.get_job("r1")["status"] == "failed"
326 assert "restart" in db.get_job("q1")["error"]
327 assert db.get_job("d1")["status"] == "done" # terminal states untouched
328
329
330 # --- real executor (cross-thread SQLite/WAL sanity) -----
331
332 def test_real_executor_completes_job(env):
333 client, db, tmp_path, mp = env
334 vid_file = _video(tmp_path)
335 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
336 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
337 pool = ThreadPoolExecutor(max_workers=1)
338 mp.setattr(m, "_get_executor", lambda: pool)
339 try:
340 job_id = client.post(
341 "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
342 deadline = time.time() + 10.0
343 body = None
344 while time.time() < deadline:
345 body = client.get(f"/api/jobs/{job_id}").json()
346 if body["status"] in ("done", "failed"):
347 break
348 time.sleep(0.05)
349 assert body is not None and body["status"] == "done"
350 assert body["result"]["status"] == "success"
351 finally:
352 pool.shutdown(wait=True)
353

```

## 28. assistant (2026-06-18T07:49:08.161Z)

Now I want to investigate a concrete defect candidate. Let me look closely at `claim\_due\_job`: it transitions `waiting`/`queued` -> `running`, but `\_run\_claimed\_job` calls `\_execute\_processing` directly. Note `claim\_due\_job` never sets `progress`. That's fine.

Let me re-examine the ``seconds_until_next_due`` query used by ``_schedule_next_drain_if_waiting``. The query: ``MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE (julianday(next_poll_at) - julianday('now')) * 86400.0 END)``. The ``julianday('now')`` uses UTC. ``next_poll_at`` was set via ``datetime('now', '+N seconds')`` which is also UTC. Consistent. Fine.

Now let me focus on what looks like a genuine bug. In ``set_job_waiting``, it does ``next_poll_at=datetime('now', ?)`` with ``f"+{max(0, int(delay_sec))} seconds"``. But ``claim_due_job`` orders by ``COALESCE(next_poll_at, created_at)``. ``next_poll_at`` is in ``datetime('now')`` format (``YYYY-MM-DD HH:MM:SS``), and ``created_at`` is ``CURRENT_TIMESTAMP`` (same format). Consistent.

Let me look at the ``_finalize_reingest`` and the watermark concern with concurrency under ``max_workers=1``. Single worker ? serialized. OK.

Let me check a subtle but real issue: in ``download_highlight``, the order of checks. ``safe_output_filename(filename)`` is called first. But FastAPI path parameter ``{filename}`` ? can it contain a slash? In Starlette, path params don't match ``/`` by default, so ``..%2F`` gets decoded. The test covers this. Fine.

Let me look at the gemini provider's ``analyze_video`` retry. After 3 upload attempts fail, returns. The processing poll: ``_processed()`` reassigns ``video_file`` via ``files.get``. If ``files.get`` raises inside ``poll_until``, it propagates as a generic Exception caught at line 96. OK.

Let me check line 100-101: ``if video_file.state.name == "FAILED"``, then ``video_file.error.message``. If ``error`` is None this would `AttributeError`. But that's an edge in the Google SDK. Let me look elsewhere for a clearer defect.

Let me examine the config ``get`` method's ``__getitem__`` and check for the ``duration_min`` falsy bug more concretely, and check how ``query_play_segments`` filters are used. Actually let me re-examine ``upload_part`` for a real race: ``_uploads_lock`` is held only to get ``st``, then released. The write happens without the lock. Under ``max_workers=1`` executor ? but uploads come via FastAPI async endpoints, not the job executor. Two concurrent PUTs to the same upload\_id could interleave. But the sequential part check (``index != st["parts"]``) makes the client serialize. This is documented as in-process single-box alpha. Not a clear defect.

Let me now look carefully at ``_get_builtin_defaults()`` merge in ``load_config``. ``merged = {**_get_builtin_defaults(), **file_data}``. This is a SHALLOW merge. If ``file_data`` has a partial section like ``{"indexing": {"motion_threshold": 0.1}}``, it REPLACES the entire ``indexing`` default dict with just ``{"motion_threshold": 0.1}``. But Pydantic then validates ``merged`` ? and since ``IndexingConfig`` has defaults for all other fields, the missing ones get filled. So shallow merge is fine because the defaults are re-applied by Pydantic. Actually wait ? ``_get_builtin_defaults()`` returns the full dict including the full ``indexing`` block, but ``**file_data`` overwrites ``indexing`` entirely with the partial. Then Pydantic fills defaults. So net effect is correct. Not a bug.

But there's a subtlety: the whole point of merging defaults first is pointless then, since Pydantic fills defaults anyway. It's harmless. Let me move on.

Let me look at the segmenters base and validators base briefly to confirm contracts, and check ``emit_indexer_report`` for the ``finally`` block interaction. Actually, let me focus on the strongest candidate I've found and verify it. Let me re-read the ``upload_complete`` ``total_parts`` check interplay with ``test_complete_with_wrong_part_count_is_409``.

Let me check one thing in ``process_video`` submit: ``req.player_pool`` and ``req.max_frames`` are passed to ``create_job``. In ``_job_to_request``, ``job.get("player_pool")`` returns the deserialized list (``get_job`` deserializes). Good. But ``claim_due_job`` calls ``self.get_job(row["id"])`` which deserializes. Good.

Now, a real concern: ``mark_job_running`` is called by ``claim_due_job``? No ? ``claim_due_job`` does its own ``UPDATE ... SET status='running'`` but does NOT set ``progress='starting'``. ``mark_job_running`` is only used in tests. Not a production bug.

Let me look at whether there's an issue with ``_drain_due_job`` being submitted on every ``/api/process`` but also re-armed via timer ? could cause the same job to run twice.

`claim\_due\_job` uses compare-and-set so double-claim is prevented. Good.

Let me verify the `WatermarkConfig` / `StitchingConfig` defaults and check the compiler signature to ensure no contract mismatch.

## 29. user (2026-06-18T07:49:08.988Z)

```
1 import os
2 import subprocess
3 import tempfile
4 import logging
5 from pathlib import Path
6 from typing import Any, Dict, List, Optional, Tuple, Union
7
8 logger = logging.getLogger(__name__)
9
10 # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11 # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12 # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13 # backend/pipeline/stitcher/, so parents[2] == backend/.
14 _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16 class VideoCompiler:
17 def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18 watermark: Optional[Any] = None):
19 self.output_dir = output_dir
20 self.end_padding = end_padding
21 self.begin_padding = begin_padding
22 # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23 # normalized to a dict (or None) so this module stays config-package-agnostic.
24 self.watermark = self._normalize_watermark(watermark)
25 os.makedirs(output_dir, exist_ok=True)
26
27 @staticmethod
28 def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29 if watermark is None:
30 return None
31 if hasattr(watermark, "model_dump"):
32 return watermark.model_dump()
33 if isinstance(watermark, dict):
34 return dict(watermark)
35 return None
36
37 @staticmethod
38 def _ff_quote(path: str) -> str:
39 """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40 filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41 slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43 The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44 option parser treats ':' as the option separator, so a Windows drive-letter
45 path like `C:/dir/font.ttf` otherwise truncates the operand at `C` and
46 the encode fails with "Invalid argument". This silently disabled the
47 on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48 return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50 @staticmethod
51 def _parse_time(val: Union[int, float, str]) -> float:
52 """Normalizes a timestamp value to float seconds.
53 Handles: float/int (pass-through), plain numeric strings ("12.4"),
```

```

54 and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55 This mirrors the parse_time logic used in gemini.py to ensure
56 the stitcher can always consume whatever format the indexer produced.""
57 if isinstance(val, (int, float)):
58 return float(val)
59 if isinstance(val, str):
60 val = val.strip()

```

### 30. assistant (2026-06-18T07:49:19.089Z)

The compiler signature matches the call sites. Now let me focus on a specific defect candidate I want to confirm: the ``compile_highlights`` endpoint matching ``s["id"]` in `req.segment_ids`` when segments have duplicate ids ? no. Let me check the ``query_play_segments`` ``duration_min`` falsy bug, which is a genuine logic issue. With ``duration_min=0.0``, ``filters.get("duration_min")`` is ``0.0`` which is falsy, so the filter is skipped ? but skipping a ``>= 0`` filter returns everything anyway, so semantically equivalent. Not a real defect.

Let me look at the ``set_job_waiting`` and ``claim_due_job`` ordering issue with the timezone. Actually let me re-examine a genuine concern: ``claim_due_job`` SELECT uses ``next_poll_at <= CURRENT_TIMESTAMP``. ``CURRENT_TIMESTAMP`` in SQLite is UTC. ``next_poll_at`` set by ``datetime('now', '+N seconds')`` is UTC. Consistent. Good.

Let me check the ``_arm_wake_timer`` ? it creates a ``threading.Timer`` that calls ``_get_executor().submit(_drain_due_jobs)``. After the executor is shut down (server stop), the timer would call submit on a shut-down executor raising ``RuntimeError``. But the timer is a daemon and process is exiting. Minor.

Let me now examine the validators registry to confirm ``run_all_with_context`` contract, and look at one more area ? the ``get_config`` endpoint returning ``global_config`` (a Pydantic model). FastAPI serializes it. Fine.

Let me look more carefully at a potential real bug in the ``upload_part`` total-size accounting when a part is exactly at the boundary, and the ``os.replace`` in ``upload_complete`` across filesystems. Actually, let me check the ``download_highlight`` MIME / ``mov`` handling ? ``safe_output_filename`` already rejected separators. Fine.

Let me step back and scan the segmenters base and validators base for the actual contract used by main.py.

### 31. user (2026-06-18T07:49:19.959Z)

```

1 from abc import ABC, abstractmethod
2 from typing import List, Dict, Any, Type, Optional
3 from backend.infrastructure.database import Database
4 from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6 class BasePlaySegmenter(ABC):
7 def __init__(self, db: Database, config: Dict[str, Any]):
8 self.db = db
9 self.config = config
10
11 @property
12 @abstractmethod
13 def name(self) -> str:
14 """Unique identifier for this segmenter."""
15 pass
16
17 @property
18 @abstractmethod
19 def description(self) -> str:
20 """Human-readable explanation of this segmenter."""
21 pass
22
23 @abstractmethod

```

```

24 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25 """
26 Processes a video to detect play segments, populates SQLite database,
27 and returns a Result.
28
29 Use isinstance(result, Failure) (or result.is_success()) to distinguish
30 "no segments found" from "something went wrong" (missing key, healthcheck
31 failure, etc.). This is the start of the standardized error/result contract.
32 """
33 pass
34
35 class SegmenterRegistry:
36 def __init__(self):
37 self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39 def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40 """Registers a new segmenter class under its name property."""
41 try:
42 temp_inst = segmenter_cls(None, {})
43 name = temp_inst.name
44 except Exception as e:
45 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46 self._segmenters[name] = segmenter_cls
47
48 def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49 return self._segmenters.get(name)
50
51 def list_available(self) -> Dict[str, str]:
52 """Returns a dict of registered segmenter names and their descriptions."""
53 res = {}
54 for name, cls in self._segmenters.items():
55 try:
56 temp_inst = cls(None, {})
57 desc = temp_inst.description
58 except Exception:
59 desc = "No description available."
60 res[name] = desc
61 return res
62
63 segmenter_registry = SegmenterRegistry()
64

```

### 32. assistant (2026-06-18T07:49:24.091Z)

Now let me confirm one suspicion about `execute\_processing`'s `result.value` if `hasattr(result, "value")` else `result`. The `Result` is `Success | Failure`. The `Failure` branch is checked first (`isinstance(result, Failure)`). If success, `result.value`. If the segmenter returns a raw list (not wrapped in Success), then `hasattr(result, "value")` is False, so `segments = result`. OK.

Let me look at the `results.py` to confirm `Failure.message` and `Success.value`.

### 33. user (2026-06-18T07:49:24.900Z)

```

1 """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3 This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5 The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6 (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8 We use simple dataclasses so callers can use isinstance checks or structural pattern

```

```

matching.
9 This is intentionally small and incremental ? we start by applying it to the segmenter
10 and provider layers (the biggest sources of silent [] returns today).
11
12 The types are generic where it makes sense. Callers can still access .value on Success
13 or .message on Failure.
14
15 This module has no dependencies on the rest of the app.
16 """
17
18 from __future__ import annotations
19
20 from dataclasses import dataclass, field
21 from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
22
23 T = TypeVar("T")
24
25
26 @dataclass
27 class Success(Generic[T]):
28 """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30 value: T
31 warnings: List[str] = field(default_factory=list)
32
33 def is_success(self) -> bool:
34 return True
35
36 def unwrap(self) -> T:
37 """Return the value. Only safe to call if you already know it's a Success."""
38 return self.value
39
40
41 @dataclass
42 class Failure:
43 """Represents a failure with a human-readable message and optional details."""
44
45 message: str
46 details: Optional[dict[str, Any]] = None
47 is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49 def is_success(self) -> bool:
50 return False
51
52 def unwrap(self) -> Any:
53 """Raise to make misuse obvious in code that assumes success."""
54 raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57 # Convenient alias for use in type hints: Result[list[dict], Failure]
58 # Result[ValueType] means Success[ValueType] | Failure
59 # (the Failure "error" type is fixed for this contract)
60 Result = Union[Success[T], Failure]
61
62
63 def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64 """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66 - ``Failure`` -> ``(None, failure)``
67 - ``Success(value=v)`` -> ``(v, None)``
68 - a bare value (legacy code that predates this contract and still returns a raw
list)
69 -> ``(value, None)``
70

```

```

71 This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72 that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73 on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74 (Success | Failure | legacy bare value)."""
75 if isinstance(result, Failure):
76 return None, result
77 value = result.value if hasattr(result, "value") else result
78 return value, None
79

```

#### 34. assistant (2026-06-18T07:49:43.738Z)

Now I have a thorough understanding. Let me investigate one final concrete concern I noticed ? the response shape mismatch in `\_execute\_processing`. The tests reference `direct\_out.get("result", {})` and `job\_out["result"]["status"]`. In `\_execute\_processing`, the returned `response` is a flat dict with `status` at the top level (e.g. `{"video\_id":..., "status":"success", ...}`). The test at line 138 does `direct\_out.get("result", {}).get("status")` ? which would be `None` for the direct path since there's no nested `result` key. The test uses `or` to tolerate this. That's a test quirk, not a code bug.

Now let me examine a real candidate carefully: in `mark\_job\_done`, `result` includes `play\_segments` which for the `\_FakeSeg` is a list of dicts ? JSON serializable. But in the real pipeline, `segments = result.value` could contain segment dicts with `metadata`. Those should be JSON-serializable. OK.

Let me check one genuinely suspicious thing: `get\_job` does `d["result"] = json.loads(d["result"]) if d["result"] else None`. If a job's `result` was stored but JSON-parse fails (corruption), `get\_job` raises (no try/except), and since `get\_job` is called inside `claim\_due\_job` after commit, the exception is caught there and returns None ? meaning a successfully-claimed job (now 'running') gets dropped. But that requires corruption. Edge.

Let me verify the most promising real defect: the `seconds\_until\_next\_due` / wake-timer using `\_MAX\_DRAIN\_WAKE\_SEC` defined AFTER the functions that... no, module-level constant is fine at runtime.

Let me check whether there is a genuine off-by-one or contract issue with `claim\_due\_job` not resetting `finished\_at`/`error` for a re-claimed waiting job. A waiting job never had `finished\_at` or `error` set. So re-claim is clean.

Let me check the most concrete reachable bug I can find: in `process\_video` (submit), if `db\_instance` is `None`. Not reachable in normal flow.

Let me verify one thing about `validate\_input\_path` + Windows. On Windows, `req.video\_path` with no `allowed\_root` only checks null bytes. So `os.path.exists` then `compute\_video\_id`. Fine.

Let me check the `gemini.py` provider once more ? line 86-87, `\_processed` reassigns the nonlocal `video\_file`, and if `files.get` raises `PolicyTimeout`... no, `poll\_until` catches nothing; `check()` raising propagates out of `poll\_until` to line 96 generic handler. But the generic handler at line 96 then does `self.\_safe\_delete(video\_file)` ? `video\_file` is still the uploaded file (the failed `get` reassignment didn't happen because the exception was raised mid-call). Actually `video\_file = self.client.files.get(...)` ? if `files.get` raises, the assignment doesn't happen, `video\_file` keeps prior value. `\_safe\_delete` works. Fine.

I've done a thorough review. Let me look at one more area I haven't fully checked ? the `loader.py` shallow merge interaction with `validate\_assignment=True` and a potential issue where `file\_data` is not a dict (e.g. JSON file containing a list or a string).

In `load\_config`, `file\_data = json.load(f)`. If the config.json contains a JSON array or scalar (e.g. `[]` or `"foo"`), `file\_data` would be a list/str, not a dict. Then `if file\_data:` is truthy, `\_unknown\_key\_paths(file\_data, AppConfig)` calls `data.items()` on a list ? `AttributeError`, which is NOT caught (the try/except only wraps the file read at lines 67-73). This would crash `load\_config`. Let me verify.

### 35. user (2026-06-18T07:49:45.274Z)

```
63 cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65 file_data: dict[str, Any] = {}
66 if cfg_path.exists():
67 try:
68 with open(cfg_path, "r", encoding="utf-8") as f:
69 file_data = json.load(f)
70 except Exception as exc:
71 # Non-fatal: continue with defaults + whatever we could parse.
72 # In production we might log here.
73 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75 if file_data:
76 unknown = _unknown_key_paths(file_data, AppConfig)
77 if unknown:
78 logger.warning(
79 "[CONFIG WARNING] %s contains keys the typed config does not "
80 "recognize (typo'd or removed knobs ? top-level extras are kept "
81 "but unread; unknown section keys are silently dropped): %s",
82 cfg_path, ", ".join(sorted(unknown)),
83)
84
85 # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86 merged = {**_get_builtin_defaults(), **file_data}
87
88 return AppConfig.model_validate(merged)
```

### 36. assistant (2026-06-18T07:49:57.952Z)

This is a real but low-severity defect: if ``config.json`` contains valid JSON that is not an object (e.g. a top-level array ``[...]``, string, or number), ``json.load`` succeeds (so the try/except at 67-73 doesn't catch it), and then ``_unknown_key_paths(file_data, AppConfig)`` calls ``data.items()`` on a non-dict, raising ``AttributeError``. Also ``{**defaults, **file_data}`` would fail for a non-mapping. This crashes ``load_config``, which runs at startup. It's an operator-error config, low severity, but it's a genuine unguarded external-data access. Let me confirm there's no validation that ``file_data`` is a dict elsewhere. There isn't.

Now let me look at one more high-value spot: ``_unknown_key_paths`` recursion with ``fields[key].annotation``. For ``Optional[X]`` fields the annotation is a ``Union``, not a ``type``, so ``isinstance(ann, type)`` guards it. Fine.

Let me now verify whether ``load_config`` non-dict crash is actually reachable in practice. The doctor writes a valid object config, but a user could hand-edit. It's reachable. I'll report it as low severity.

Let me check one more genuinely impactful thing ? does ``_drain_due_jobs`` get submitted to the executor that uses ``db_instance`` from a worker thread while ``db_instance`` could be None at import? No, set in ``main()``.

Let me do a final targeted check on the ``claim_due_job`` SELECT ordering correctness vs a potential issue: a ``queued`` job has ``next_poll_at = NULL``, so ``COALESCE(next_poll_at, created_at)`` orders by ``created_at``. A ``waiting`` job uses ``next_poll_at``. These are compared in the same ORDER BY. Both in ``YYYY-MM-DD HH:MM:SS`` UTC string format, so lexicographic ordering works. Fine.

I've completed a thorough adversarial review. Let me verify the one config defect by examining if there's a test for non-dict config to be sure it's not already handled.

### 37. user (2026-06-18T07:49:58.821Z)

Found 3 files  
tests/test\_watermark.py

tests\test\_generate\_highlights.py  
tests\test\_config.py

### 38. user (2026-06-18T07:50:01.683Z)

```
1 """Basic tests for the typed configuration package.
2
3 These ensure:
4 - Defaults are sensible and come from the models.
5 - File overrides work.
6 - Old config files with extra keys are tolerated.
7 - save_default_config produces loadable output.
8 """
9 import json
10 import tempfile
11 import os
12
13
14 from backend.config import load_config, save_default_config, AppConfig
15
16
17 def test_load_default_config():
18 # Force load without the project's config.json so we test the model built-in
19 defaults
20 cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
21 assert isinstance(cfg, AppConfig)
22 assert cfg.sport == "badminton"
23 assert cfg.indexing.default_segmenter == "yolo_hybrid"
24 assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
25 assert cfg.indexing.ai_max_workers == 5
26
27 def test_logging_config_default_quiets_http():
28 """verbose_http defaults to False so run.log is quiet unless explicitly turned
29 up."""
30 cfg = load_config("/nonexistent/path/12345/config.json")
31 assert cfg.logging.verbose_http is False
32
33 def test_skip_ai_handoff_default_off():
34 """Fully-local mode is opt-in: skip_ai_handoff defaults False so the AI handoff
35 runs."""
36 cfg = load_config("/nonexistent/path/12345/config.json")
37 assert cfg.indexing.skip_ai_handoff is False
38
39 def test_skip_ai_handoff_file_override():
40 with tempfile.TemporaryDirectory() as td:
41 p = os.path.join(td, "skip.json")
42 with open(p, "w") as fh:
43 json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
44 cfg = load_config(p)
45 assert cfg.indexing.skip_ai_handoff is True
46
47
48 def test_logging_config_file_override():
49 with tempfile.TemporaryDirectory() as td:
50 p = os.path.join(td, "logging.json")
51 with open(p, "w") as fh:
52 json.dump({"logging": {"verbose_http": True}}, fh)
53 cfg = load_config(p)
54 assert cfg.logging.verbose_http is True
55
56
57 def test_fusion_config_defaults_reproduce_control():
```

```

58 """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask.
59
60 Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was
REVERTED
61 (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on
the
62 served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies).
See
63 docs/QUALITY_ITERATIONS.md and the served litmus in
tests/test_served_gate_a_regression.py.
64 """
65 cfg = load_config("/nonexistent/path/12345/config.json")
66 f = cfg.indexing.fusion
67 assert f.substrate == "wasb"
68 assert f.cue_flags == {} # person cue OFF
69 assert f.min_crossings == 0 # Gate A OFF (default) ? #146 served flip
reverted
70 assert f.min_players == 0 # Gate B OFF
71 assert f.court_polygon is None # no mask
72 assert f.velocity_threshold == 0.02
73
74
75 def test_fusion_config_gate_a_opt_in_via_min_crossings():
76 """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only
operating
77 point), while the served default stays OFF (#146 flip reverted)."""
78 with tempfile.TemporaryDirectory() as td:
79 p = os.path.join(td, "optin.json")
80 with open(p, "w") as fh:
81 json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
82 cfg = load_config(p)
83 assert cfg.indexing.fusion.min_crossings == 2 # Gate A ON only when explicitly
set
84
85
86 def test_served_default_segmenter_is_gemini():
87 """The shipped config.json served default is the prior detector (gemini) ? the #146
flip
88 to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
89 repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
90 cfg = load_config(os.path.join(repo_root, "config.json"))
91 assert cfg.indexing.default_segmenter == "gemini"
92 # ...and the served fusion default keeps Gate A OFF (no served gating).
93 assert cfg.indexing.fusion.min_crossings == 0
94
95
96 def test_fusion_config_file_override():
97 with tempfile.TemporaryDirectory() as td:
98 p = os.path.join(td, "fusion.json")
99 with open(p, "w") as fh:
100 json.dump({"indexing": {"fusion": {
101 "substrate": "tracknet", "min_crossings": 2,
102 "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
[0.5, 0.9]],
103 }}, fh)
104 cfg = load_config(p)
105 f = cfg.indexing.fusion
106 assert f.substrate == "tracknet"
107 assert f.min_crossings == 2
108 assert f.cue_flags == {"person": True}
109 assert len(f.court_polygon) == 3
110 # An old config without a fusion block still loads (defaults applied).
111 assert cfg.indexing.wasb.velocity_threshold == 0.02
112

```

```

113
114 def test_file_override():
115 with tempfile.TemporaryDirectory() as td:
116 p = os.path.join(td, "override.json")
117 data = {
118 "indexing": {
119 "default_segmenter": "gemini",
120 "ai_max_workers": 3,
121 },
122 "validation": {
123 "min_blur_threshold": 42.0,
124 },
125 }
126 with open(p, "w") as f:
127 json.dump(data, f)
128
129 cfg = load_config(p)
130 assert cfg.indexing.default_segmenter == "gemini"
131 assert cfg.indexing.ai_max_workers == 3
132 assert cfg.validation.min_blur_threshold == 42.0
133 # Unspecified keys fall back to defaults
134 assert cfg.sport == "badminton"
135
136
137 def test_extra_keys_tolerated():
138 """Old or future keys in config.json must not break loading."""
139 with tempfile.TemporaryDirectory() as td:
140 p = os.path.join(td, "with_extra.json")
141 data = {
142 "sport": "badminton",
143 "indexing": {"default_segmenter": "yolo_hybrid"},
144 "some_future_key": "ignored_for_now",
145 "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
146 }
147 with open(p, "w") as f:
148 json.dump(data, f)
149
150 cfg = load_config(p)
151 assert cfg.indexing.default_segmenter == "yolo_hybrid"
152 assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
153
154
155 def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
156 """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
157 section keys are silently dropped, so without this warning a typo just makes
158 the knob a no-op (the audit's 'silently dropped config' class)."""
159 p = tmp_path / "typo.json"
160 p.write_text(json.dumps({
161 "some_future_key": 1, # top-level extra
162 "indexing": {"velocty_threshold": 0.5}, # typo'd section key
163 "validation": {"relevance": {"unknown_gate": True}}, # doubly nested
164 }))
165 import logging
166 with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
167 cfg = load_config(str(p))
168 assert isinstance(cfg, AppConfig) # still non-fatal
169 joined = " ".join(r.message for r in caplog.records)
170 assert "some_future_key" in joined
171 assert "indexing.velocty_threshold" in joined
172 assert "validation.relevance.unknown_gate" in joined
173
174
175 def test_recognized_config_does_not_warn(tmp_path, caplog):
176 p = tmp_path / "clean.json"
177 p.write_text(json.dumps({

```

```

178 "sport": "badminton",
179 "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180)))
181 import logging
182 with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
183 load_config(str(p))
184 assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
185
186
187 def test_save_default_config_roundtrip():
188 with tempfile.TemporaryDirectory() as td:
189 p = os.path.join(td, "saved.json")
190 saved = save_default_config(p)
191 assert str(saved) == p
192 assert os.path.exists(p)
193
194 cfg = load_config(p)
195 assert cfg.indexing.default_segmenter == "yolo_hybrid"
196 assert cfg.validation.min_blur_threshold == 20.0
197
198
199 def test_model_is_the_source_of_truth():
200 """Changing a default in the model should be reflected without touching json."""
201 # This is a documentation test more than anything.
202 cfg = AppConfig()
203 assert cfg.indexing.tracknet.velocity_threshold == 0.02
204
205
206 def test_output_dir_field_default_and_override():
207 """output_dir now lives on OutputConfig (was a write-only runtime hack), so
208 /api/compile and the CLI resolve the same value. Default + file override + the
209 runtime mutation that backend/main.py::main performs for --output-dir."""
210 cfg = AppConfig()
211 assert cfg.output.output_dir == "output" # built-in default
212
213 # The CLI override path: main() assigns args.output_dir onto the typed model.
214 cfg.output.output_dir = "/tmp/custom_out"
215 assert cfg.output.output_dir == "/tmp/custom_out"
216
217 # File override is honored on load.
218 with tempfile.TemporaryDirectory() as td:
219 p = os.path.join(td, "out_override.json")
220 with open(p, "w") as f:
221 json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
222 loaded = load_config(p)
223 assert loaded.output.output_dir == "reels"
224 assert loaded.output.db_name == "x.db"
225
226 def test_config_shallow_section_replacement():
227 """Positive test for Item 2 (config loader): shallow section replacement.
228
229 File-provided 'indexing' section replaces the defaults 'indexing' section entirely
230 (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
231 """
232 with tempfile.TemporaryDirectory() as td:
233 p = os.path.join(td, "shallow.json")
234 with open(p, "w") as fh:
235 json.dump({"indexing": {"default_segmenter": "motion"}}, fh)
236 cfg = load_config(p)
237 assert cfg.indexing.default_segmenter == "motion"
238 # Pydantic defaults fill the rest of the (replaced) section
239 assert cfg.indexing.ai_max_workers == 5
240
241

```

```

242 def test_config_unknown_key_paths_warning_and_tolerate():
243 """Positive test for Item 2: _unknown_key_paths warning for unknown top-level keys,
244 but tolerated (extra='allow'), and unknown section keys dropped silently.
245 """
246 with tempfile.TemporaryDirectory() as td:
247 p = os.path.join(td, "unknown.json")
248 with open(p, "w") as fh:
249 json.dump({
250 "indexing": {"default_segmenter": "motion"},
251 "completely_unknown_top": "bar",
252 "validation": {"unknown_in_section": 99} # dropped
253 }, fh)
254 cfg = load_config(p)
255 assert cfg.indexing.default_segmenter == "motion"
256 # tolerated, no crash
257
258
259 def test_get_returns_dict_for_nested_sections_legacy_compat():
260 """Regression: legacy consumers (validators/segmenters) do
261 ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
262 .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
263 chained access raises 'X object has no attribute get' on every real run.
264
265 This is the original strong legacy test (restored per #160 review). It covers the
266 full set of footgun guards: dict sections, nested dicts, scalar leaves with
defaults,
267 missing key fallbacks, and leaf keys discoverable inside sections.
268 """
269 cfg = load_config("/nonexistent/123/config.json")
270 validation = cfg.get("validation", {})
271 assert isinstance(validation, dict)
272 assert validation.get("min_resolution_width") == 1280
273 # nested-of-nested is also dict-accessible
274 assert isinstance(validation.get("relevance"), dict)
275 assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
276
277 # scalar leaves are returned unchanged; a present scalar ignores the default arg;
278 # a missing key falls back to the default
279 assert cfg.get("ai_provider") == "gemini"
280 assert cfg.get("ai_provider", "ignored-default") == "gemini"
281 assert cfg.get("does_not_exist", "fallback") == "fallback"
282
283 # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
284 # list), so a future top-level section's keys would be discoverable too
285 assert cfg.get("min_resolution_width") == 1280 # lives inside `validation`
286 assert cfg.get("begin_padding") == 0.5 # lives inside `stitching`
287
288 # legacy chained access still works
289 assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
290
291
292 def test_config_roundtrip_and_typed_reaches_pipeline(monkeypatch):
293 """Positive for Item 2: save_default_config + load_config round-trips;
294 typed AppConfig is what reaches pipeline code (e.g. main uses global_config as
AppConfig).
295 """
296 with tempfile.TemporaryDirectory() as td:
297 p = os.path.join(td, "roundtrip.json")
298 save_default_config(p)
299 cfg = load_config(p)
300 assert isinstance(cfg, AppConfig)
301 assert cfg.sport == "badminton"
302
303 # positive that typed is used in pipeline code (global_config is set at startup in
main());

```

```

304 # monkeypatch here so the test does not depend on server initialization)
305 import backend.main as main_module
306 from backend.main import get_config
307 monkeypatch.setattr(main_module, "global_config", cfg)
308 assert isinstance(get_config(), AppConfig)
309

```

### 39. assistant (2026-06-18T07:50:10.855Z)

The tests confirm no coverage for a non-dict top-level config JSON. My analysis of the ``load_config`` non-dict crash holds, but it's a low-severity operator-error edge.

I've completed a thorough adversarial review of all target files. Let me consolidate. The most defensible finding is the ``load_config`` non-dict crash. Let me reconsider severity and whether there are any higher-impact reachable defects I may have under-weighted.

One more re-examination of ``upload_part``: the ``st["bytes"]`` is updated AFTER the write loop under the lock, using ``received`` (this part's total). But if ``overflow`` triggered, ``_abort_upload`` already removed ``st`` from ``_uploads``. Then the code proceeds to ``with _uploads_lock: st["parts"] += 1`` ? wait, no. If ``overflow``, it raises `HTTPException` BEFORE reaching the ``st["parts"] += 1`` block. So the mutation is skipped. Correct.

But there is a subtle real issue: when overflow occurs mid-part, ``_abort_upload`` is called which pops ``st`` and removes the tmp file. But the ``with open(st["tmp_path"], "ab")`` block has already written some chunks before the overflow break. ``_abort_upload`` removes the file, so no leak. Correct.

Let me reconsider the ``seconds_until_next_due`` returning ``None`` when ``row`` is a Row but ``row["secs"]`` is None (no waiting jobs ? MIN over empty set is NULL). Returns None ? ``_schedule_next_drain_if_waiting`` returns without arming. Correct.

My review is complete. The single concrete defect is the non-dict config crash. Given the precision-over-recall mandate, I'll report that one finding with appropriate (low) severity and confidence.

### 40. user (2026-06-18T07:50:20.790Z)

Structured output provided successfully

### 41. assistant (2026-06-18T07:50:33.204Z)

I completed a thorough adversarial review of the AREA files (read each target and its matching tests before asserting).

The only real, reachable defect I'm confident in:

`**`backend/config/loader.py:75-86` ? `load_config` crashes at startup on valid-but-non-object JSON.**` The `try/except` wraps only the file read/parse (lines 67-73), so a syntactically valid ``config.json`` containing a top-level array/string/number passes ``json.load``, then ``_unknown_key_paths(file_data, AppConfig)`` calls ``_items()`` on a non-dict ? unhandled ``AttributeError`` (and ``{**defaults, **file_data}`` would also fail). This propagates out of the startup path (``main()`` ? ``load_app_config``), contradicting the loader's documented "missing/unreadable ? defaulted, non-fatal" contract. Low severity (operator-error config), high confidence; one-line ``isinstance(file_data, dict)`` guard fixes it. Details (file/line/trigger/fix) are in the StructuredOutput.

Areas I checked and found clean (no findings):

- ``backend/utils/hashing.py`` (streamed MD5, single source of truth) and ``backend/utils/paths.py`` (traversal/null-byte guards; ``resolve_under_root`` handles cross-drive Windows ValueError).
- ``backend/infrastructure/database.py`` ? migration ladder v1?v5 is re-runnable (idempotent ADD COLUMN swallows only "duplicate column name"); UPSERT semantics avoid the REPLACE cascade footgun; ``claim_due_job`` uses compare-and-set on ``(id, status)`` so concurrent/timer-triggered drains can't double-claim; UTC consistency holds between ``datetime('now', ...)``, ``CURRENT_TIMESTAMP``, and ``julianday('now')``.

- ``backend/main.py`` ? submit-time fast-fail validation, destroy-AFTER-confirm watermark/prune logic (verified against the raising/empty/failure branches and their tests), upload part sequencing + abort cleanup, and download traversal guards all check out.
- ``backend/providers/`` ? oversize-skip, 3x upload retry, bounded poll + per-attempt timeout/retry via ``execution_policy`` are sound.

The shallow config-section merge (``{**defaults, **file_data}``) is intentional and safe (Pydantic re-fills section defaults), and is covered by an explicit positive test ? not a defect.